Visual Programming

Dr Hedia ZARDI

H.zardi@qu.edu.sa

Chapter 2: Graphical interfaces

Dr Hedia ZARDI

H.zardi@qu.edu.sa

Outline of Chapter 2

- Graphical interfaces
- AWT
- SWING
- Create an interface
- The window JFrame
- The panel JPanel
- Adding components
- Component distributions
- Event management
- Adapters

Graphical interfaces

- A graphical interface (windows, buttons, menus…) requires a lot of components and lines of code.
- It is interesting to separate the graphical interface part from the rest of the program. The object approach is well suited.
- Java offers 2 systems for creating graphical interfaces:
  - the oldest is called **AWT** (**TO**bstract **W**indow **T**oolkitor "abstract windowing library"); java.awt package
  - the most recent (since Java 2) is called **Swing**. It is an improvement of AWT; packagejavax.swing.
- **Swing** is a set of classes used to create objects representing graphic components.

AWT

- **AWT: Abstract Window Toolkit**
  - Package: java.awt
  - Uses components **heavy** using a lot of system resources
  - Linked directly to the possibilities of the graphical user interface of the local platform.
  - Least Common Denominator

    →If not natively available on one of the platforms Java, then not available on all platforms Java
  - Simple Component Set
  - Different display depending on the platform

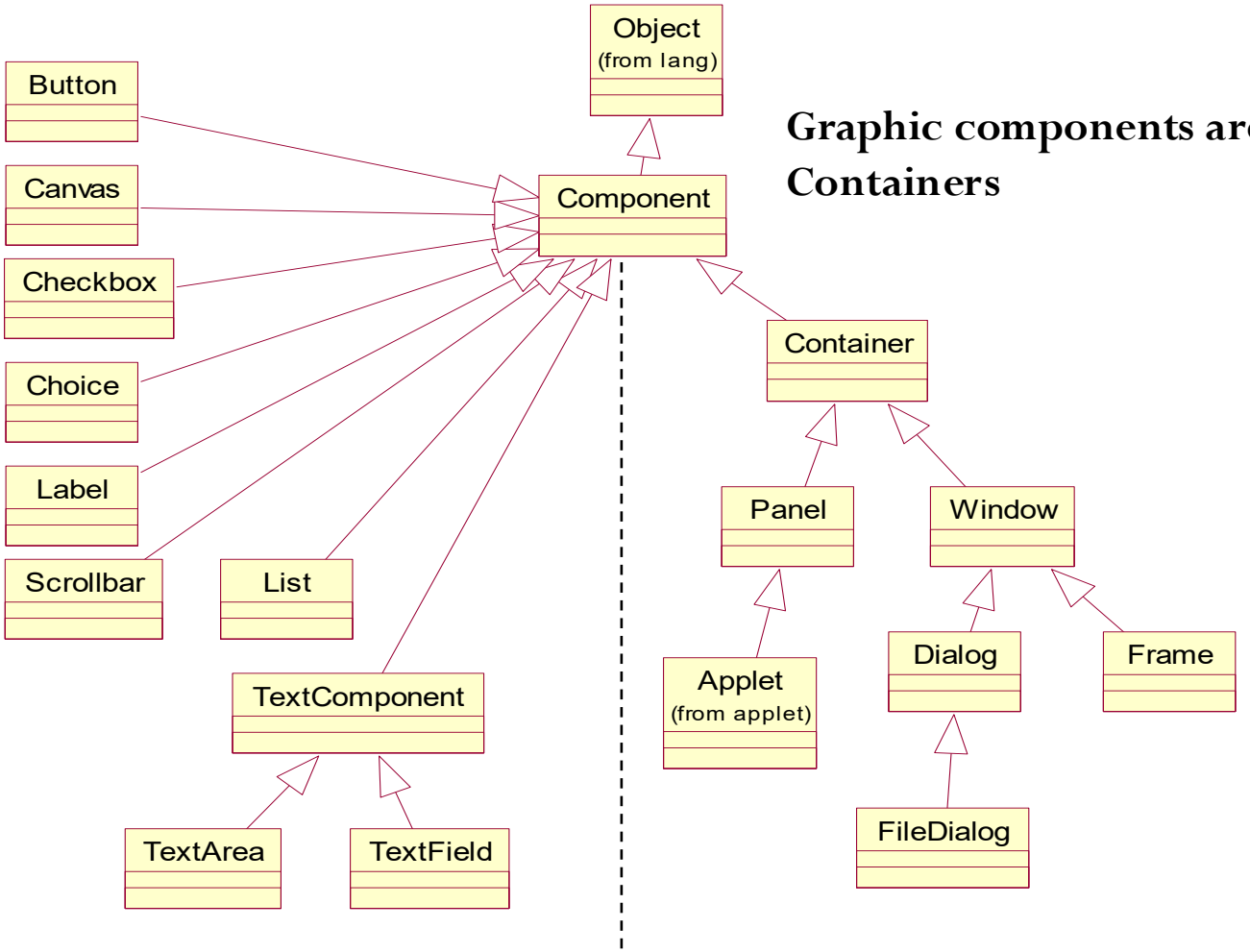
Containers and components

- A graphical interface in Java is an assembly **containers (Container) and components (Component).**

- **A component is a "visible" part of the Java user interface.**
  - It is a subclass of the abstract class **java.awt.Component.**
  - Example: buttons, text or drawing areas, etc.

- **A container is a space in which you can position several components.**
  - Class subclass **java.awt.Container**.
  - The Container class is itself a subclass of the Component class
  - Example: windows, applets, etc.

Inheritance hierarchy of the main elements of graphical interfaces in Java



Graphic components are placed in Containers

Containers and components

- The two most common containers are the **Frame** and the **Panel**.
- a **Frame** represents a high-level window with a title, border, and resizing angles.
  - Most applications use at least one Frame as a starting point for their GUI.
- a **Panel** does not have a clean appearance and cannot be used as a stand-alone window.
  - Panels are created and added to other parts in the same way as components such as buttons.
  - Panels can then redefine their own layout to contain other components themselves.

Swing

- The library **Swing** is a new library of graphics components for Java.
  - Swing is integrated with Java 1.2. (Java 2)
  - Swing can be downloaded separately for use with earlier versions of Java (1.1.5+)
- This library is added to the one that was used until then (**AWT**) for compatibility reasons.
  - Swing, however, overlaps in many cases with **AWT**.
  - Sun's ambition is that, gradually, the developers realize all their interfaces with Swing and drop the old ones. **API** graphics.

Swing

- a heavy graphics component (*heavyweight GUI component*) relies on the local window manager, that of the machine on which the program is running.
  - AWT only has heavy components.

Swing

- Example :
  - A button of type java.awt.Button integrated in a Java application on the Unix platform is represented by a real Motif button (called its peer - *peer* in English).
  - Java communicates with this Motif button using the Java Native Interface. This communication incurs a cost.
  - This is why this button is called a heavy component.

Swing

- a lightweight graphics component (in English, *lightweight GUI component*) is a graphical component independent of the local window manager.

  - A lightweight component looks like a local window manager component but is not: a lightweight component emulates local window manager components.

  - A light button is a rectangle drawn on a canvas that contains a label and responds to mouse events.

  - All Swing components, **excepted** **JApplet**, **JDialog**, **JFrame** and **JWindow** are lightweight components.

Swing package view

- **Basic components**
  - JApplet
  - JButton
  - JCheckBox
  - JRadioButton
  - JToggleButton
  - JComboBox
  - JList
  - JSlider
  - JTable
  - JTree
  - JProgessBar
  - JSpinner



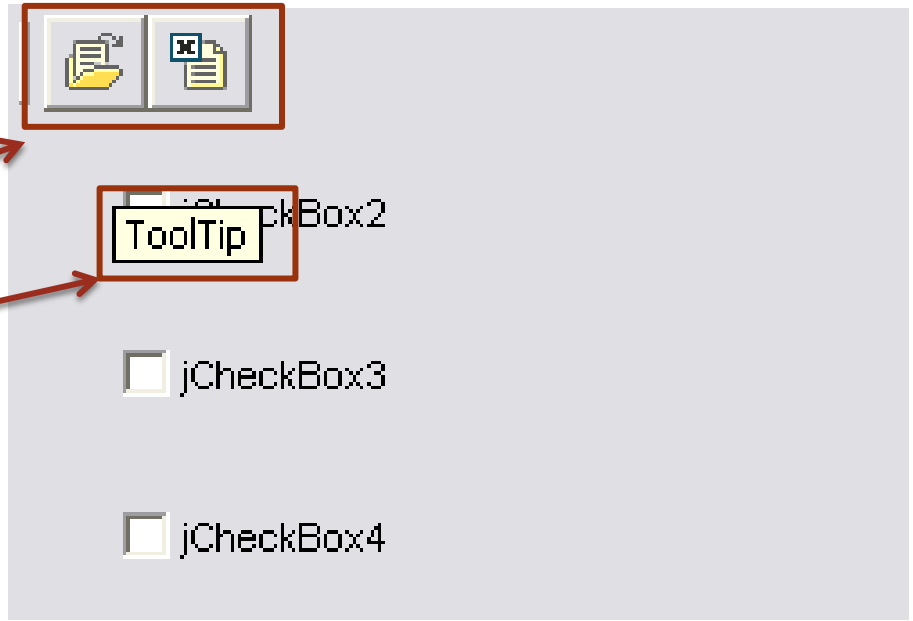
Swing package view

- **Menus, Toolbar and Info Bubbles**
  - JMenuBar
  - JMenu
  - JMenuItem
  - JCheckBoxMenuItem
  - JRadioButtonMenuItem
  - JPopupMenu
  - JToolBar
  - JToolTip



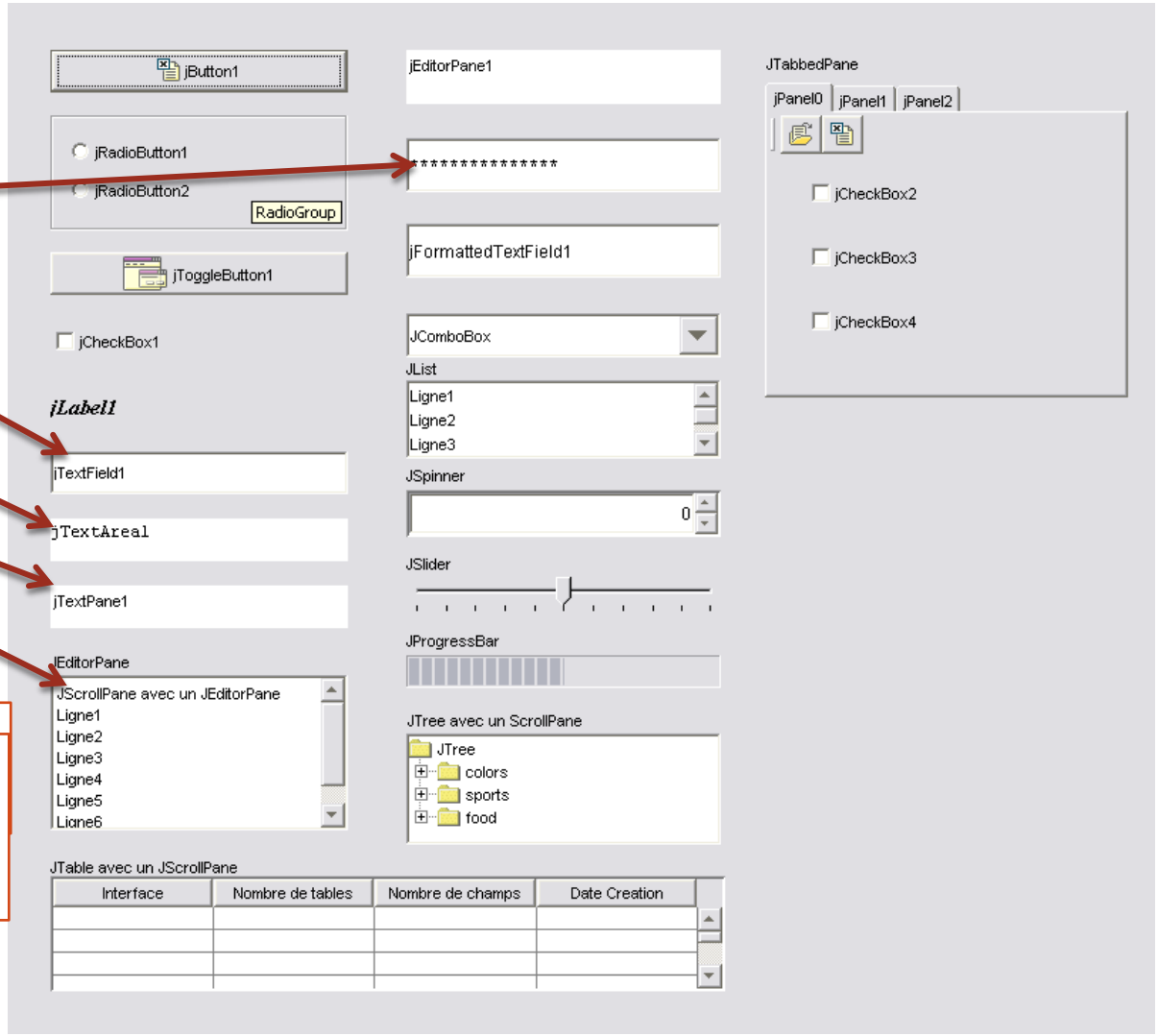
Swing package view

- **Menus, Toolbar and Info Bubbles**
  - JMenuBar
  - JMenu
  - JMenuItem
  - JCheckBoxMenuItem
  - JRadioButtonMenuItem
  - JPopupMenu
  - JToolBar
  - JToolTip



Swing package view

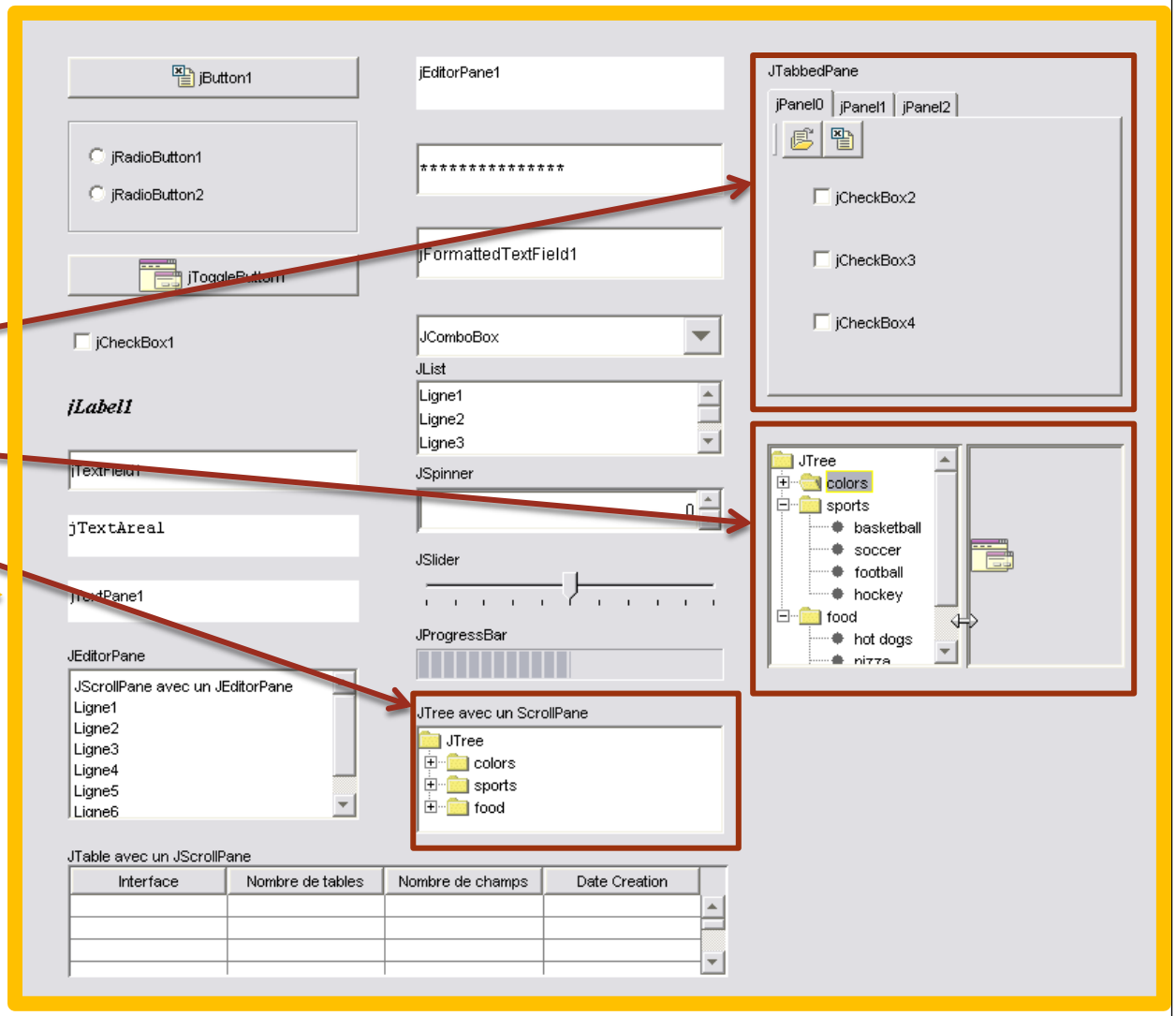
- **Text components**
  - JPasswordField
  - JTextField
  - JTextArea
  - JTextPane
  - JEditorPane



JEditorPane allows you to edit content of different kinds: Text, HTML, etc

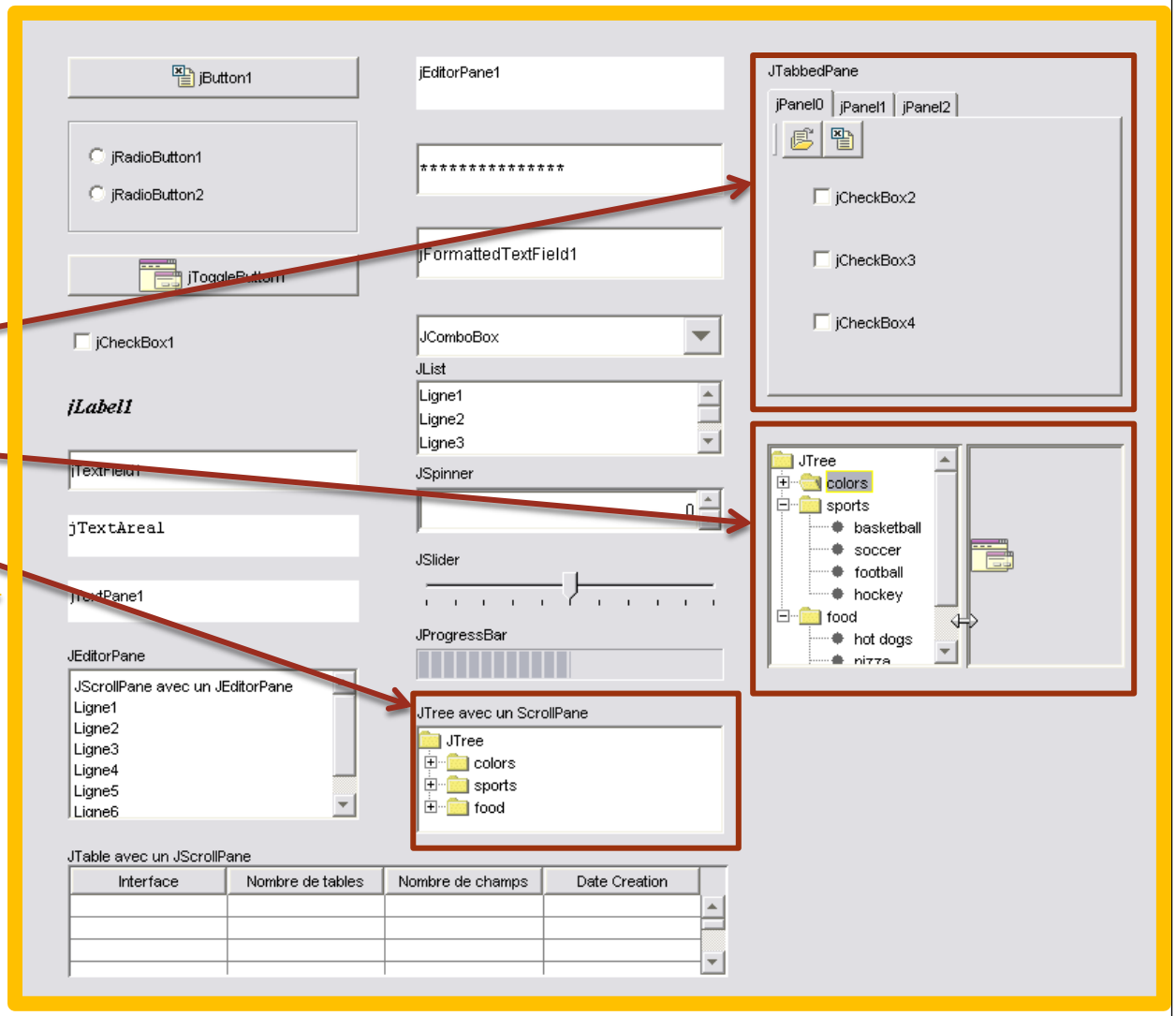
Swing package view

- **Containers**
  - JOptionPane
  - JDialog
  - JTabbedPane
  - JSplitPane
  - JScrollPane
  - JFrame
  - JInternalFrame
  - JDesktopPane
  - JWindow



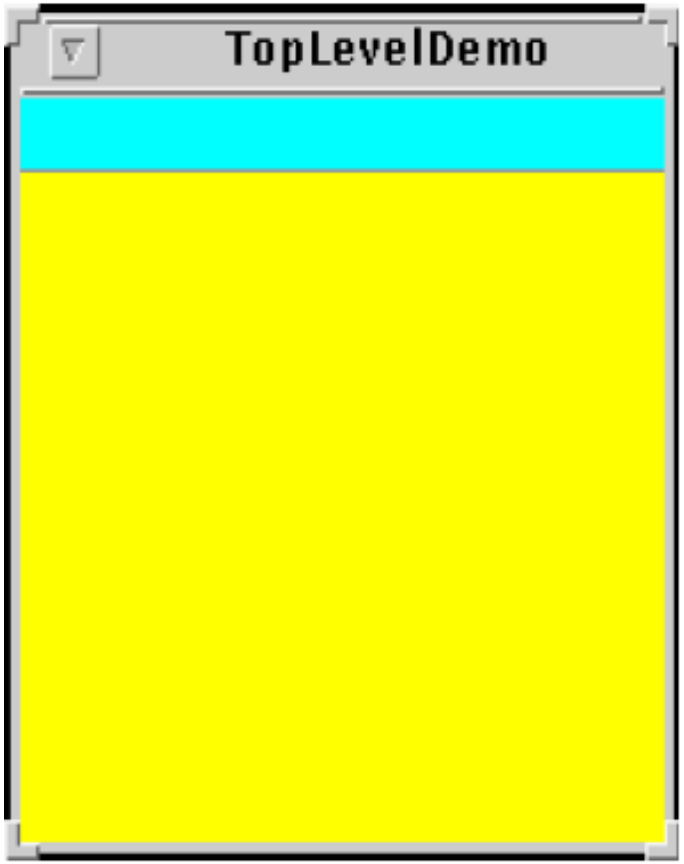
Swing package view

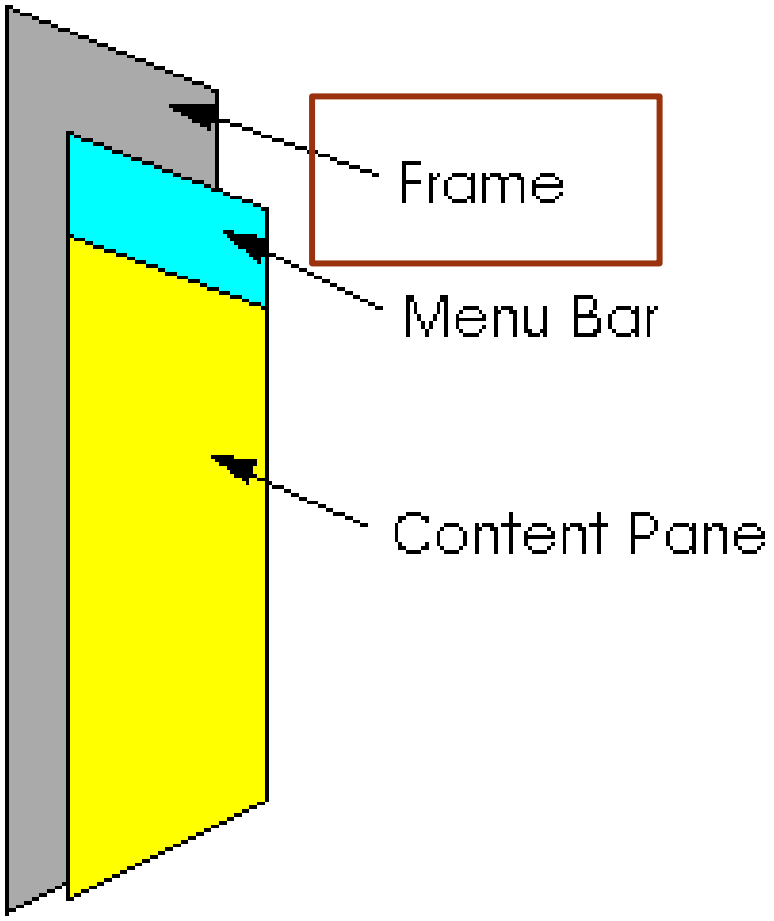
- **Containers**
  - JOptionPane
  - JDialog
  - JTabbedPane
  - JSplitPane
  - JScrollPane
  - **JFrame**
  - JInternalFrame
  - JDesktopPane
  - JWindow



Create an interface

- **The window JFrame**





The window JFrame

- To create a graphics window, we have, in the package ***javax.swing***, a standard class named ***JFrame*** possessing:
  - A constructor without arguments:
    - JFrame fen = new JFrame();
  - If we limit ourselves to that, **nothing will appear on the screen**.
  - You must request the display of the window by calling one of these two methods:
    - fen.setVisible(true);
    - fen.show();

The window JFrame

- **Other essential functions for creating a window:**
  - By default, the size of a window is zero. To define the dimensions on the screen, use the function:
    - fen.setSize(l, h);
      - h: the height in pixels;
      - L: the width in pixels;
  - **Example:**

    fen.setSize(300,150);

The window JFrame

- **Other essential functions for creating a window:**
  - In general, we will choose to display a specific text in the title bar.
    - We then use:
      - fen.setTitle(String Message);
  - **Example:**
    - fen.setTitle("My first window");

The window JFrame

- The user can handle this window like any graphics window in software:
- Resize it
- Move it
- Reduce it to an icon.

- **Warning**, closing a window of type *JFrame* born not terminate the program, but simply make the window invisible (as if calling the method ***setVisible (false)).***

The window JFrame

- For closing the window to end the program, add:
  - *setDefaultCloseOperation (JFrame.EXIT_ON_CLOSE);*

- To close the window without terminating the program, add:
  - *setDefaultCloseOperation (JFrame.DISPOSE_ON_CLOSE);*

- The effect of this instruction is only visible if you have opened more than one window in your program.

The window JFrame

- **Creating a custom window class:**
  - In the previous example, we simply created an object of type *JFrame* and we used the functionality present in this class.
  - For our program to be of interest, additional functionality must be associated with it: the window must be able to react to certain events.
  - For this, we will need to define our own class derived from *JFrame* and create an object of this new type.

The window JFrame

- **Creating a custom window class:**

```
import javax.swing. *;

public class Window extends JFrame{
 public Window()
 {
 setSize(300,150);
 setTitle("My first window");
 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}

 public static void main (String [] args) {
 Window premFen = new Window();
 premFen. setVisible(true);
 }
}
```

The window JFrame

- **Reposition and resize a window:**
  - You can use the function:
    - setBounds(px, py, l, h)
    - With:
      - px and py: the coordinates of the pixel which presents the new position of the window.
      - h and l: the new dimensions of the window.

The window JFrame

- **Reposition and resize a window:**

```
import javax.swing. *;

public class Window extends JFrame{
 public Window()
 {
 setSize(300,150);
 setTitle("My first window");
 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}

 public static void main (String [] args) {
 Window premFen = new Window();
 premFen. setVisible(true);
 premFen.setBounds(250,250,300,200);
 }
}
```

The window JFrame

- **Reposition and resize a window:**
- If we want to position the window in relation to the size of the screen:

  - You must use a tool kit (Toolkit) allowing you to load the dimensions, icons and all the necessary tools from the Operating System (Java.awt package).

The window JFrame

- **Reposition and resize a window:**
- **Example**:

```
import javax.swing. *;

public class Window extends JFrame{
 public Window()
 {
 setTitle("My first window");
 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
}
 public static void main (String [] args) {
 Window premFen = new Window();
 premFen. setVisible(true);
Toolkit kit =Toolkit.getDefaultToolkit();
Dimension screenSize = kit.getScreenSize();
int H = screenSize.height; // Height
int W = screenSize.width; //Width
premFen.setBounds(W / 4, H / 4, W / 2, H / 2);
Picture img = kit.getImage("icon.gif");
this.setIconImage(img);
 }
}
```

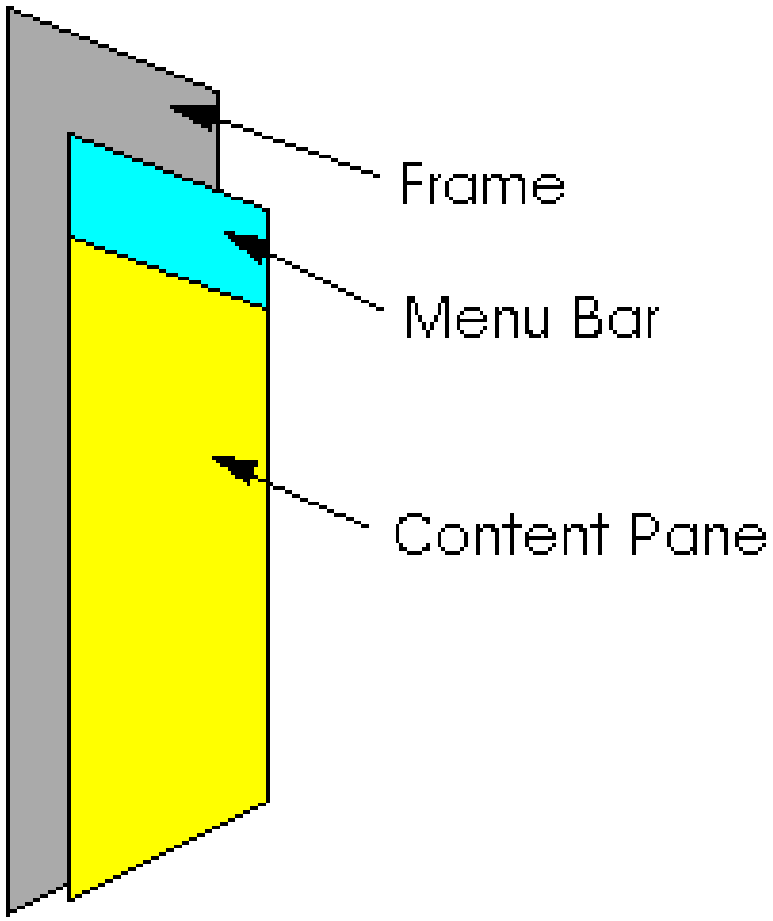
Repositioning and resizing relative to the screen

Changing the window icon

The panel JPanel

- He **not possible** to directly introduce a component in a window (***JFrame)*.**
- ***JFrame*** has a structure complex. It is formed by a superposition of several elements, in particular a root, a content,etc .
- The method ***getContentPane*** of style ***JFrame*** provides the reference to this content of type Container:
  - ***Container c = getContentPane();***

  Or
  - **Jpanel c = new JPanel(); c = (JPanel)getContentPane();**



The panel Jpanel: Addition of components

- the different components are to be added in the content part with method ***add*** .
  - **c.add (component);**
  - **c.remove(making up);** for deletion.

- But the method **add** always add the component in the same place in the container.

- So when you want to add several components to the panel, only the last component appears .

The panel Jpanel: Addition of components

```
class Window extends JFrame
{
        private JButton button1, button2;
        private JLabel label1, label2;
        private JTextField text1, text2;

        public Window()
        {
                setSize(400,300);
                setTitle("A window with components");

                // create components
                button1 = new JButton("Button 1");
                button2 = new JButton("Button 2");
                label1 = new JLabel("Label 1");
                label2 = new JLabel("Label 2");
                text1 = new JTextField("Text box 1");
                text2 = new JTextField("Text box 2");
```

The panel Jpanel: Addition of components

```
JPanel c = (JPanel)getContentPane();

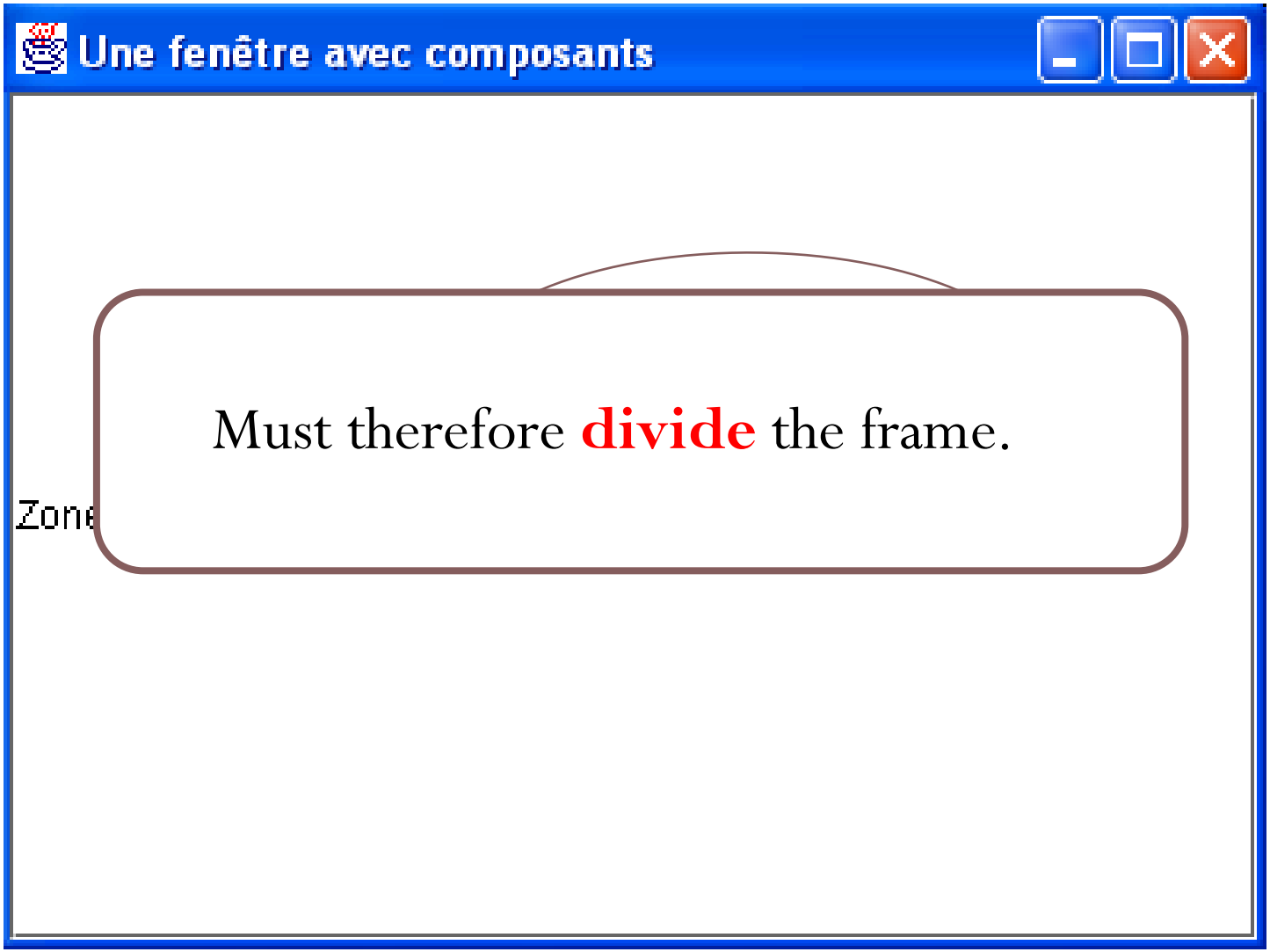
// add the components
        c.add (button1);
        c.add (label1);
        c.add (text1);
        c.add (button2);
        c.add (label2);
        c.add (text2);

} // end of constructor of the class Window

    public static void main (String args[])
    {
        Window premFen = new Window ();
        premFen.show();
    }

} // end of the class Window
```

The panel Jpanel: Addition of components



Absolute positioning

- To place components at **a specific location,** we must indicate that we do not use ***LayoutManager.***

ContainerObject.setLayout(null);

- And we indicate the coordinates and size of each component

  Making up.setBounds(x, y, width, high);

  - x is the displacement relative to the left of the container
  - y is the displacement relative to the top of the container

Presentation managers

- Each container is associated with a presentation manager (**GP**) or layout manager (***layout manager***)
- The GP controls the location and size of components each time they are displayed.
- The rearrangement of components in a container takes place during:
  - the modification of its size,
  - changing the size or moving one of the components.
  - adding, showing, removing or hiding a component.

Presentation managers

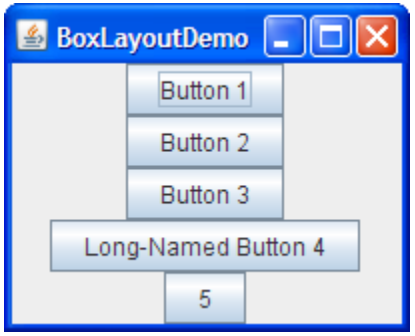
- Any container has a default GP.
  - Any instance of ***Container*** references an instance of the class ***LayoutManager*** of the package ***java.awt.***
  - It is possible to change it using the method ***setLayout().***
- All the GPs are classes of ***java.awt*** who inherit from ***LayoutManager.***

Presentation managers

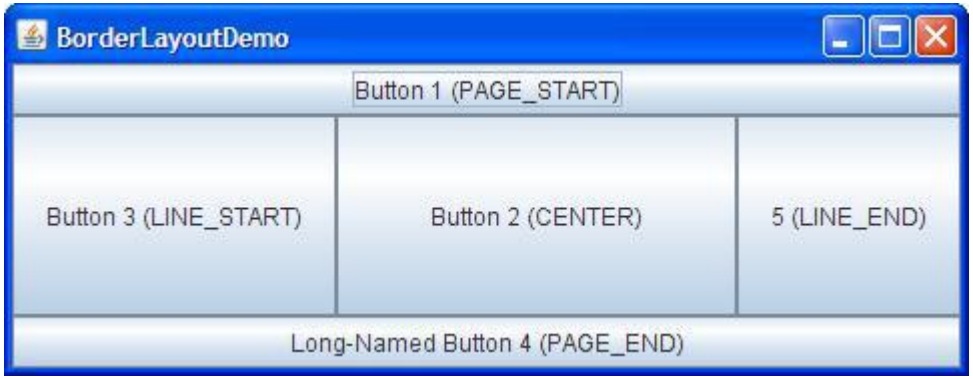
- To position a component, we have several predefined positions. These positions that are offered by Java are:
  - BorderLayout
  - BoxLayout
  - CardLayout
  - FlowLayout
  - GridBagLayout
  - GridLayout
- The GPs default are:
  - the BorderLayout for JFrame and his descendants
  - the FlowLayout for JPanel and his descendants (JApplet, etc.)

Presentation managers

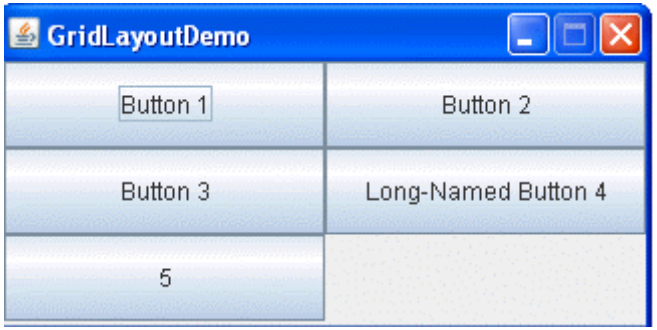
BoxLayout



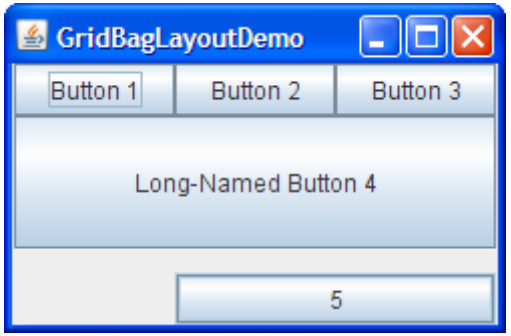
BorderLayout



GridLayout



GridBagLayout



FlowLayout

- FlowLayout organizes components in rows, left to right, then top to bottom, using the preferred size (preferredSize) of each component.

- FlowLayout places as many components as possible online, before starting over on a new row.

- Alignment in a **FlowLayout** can be on the left (FlowLayout.LEFT), To law (FlowLayout.RIGHT) Where centered (FlowLayout.CENTER)

  - by default, components are centered within the area allocated to them.
  - a horizontal or vertical spacing between two components is defined using the methods ***setHgap() and setVgap()***.

FlowLayout

- the **FlowLayout** actually and effectively hides the components that do not fit in the frame.

- the **FlowLayout** is only of interest when there are few components in the container.

- **Example:**
  - JPanel p = (JPanel) this.getContentPane();

```
p.setLayout(new FlowLayout(FlowLayout.LEFT));
```

FlowLayout

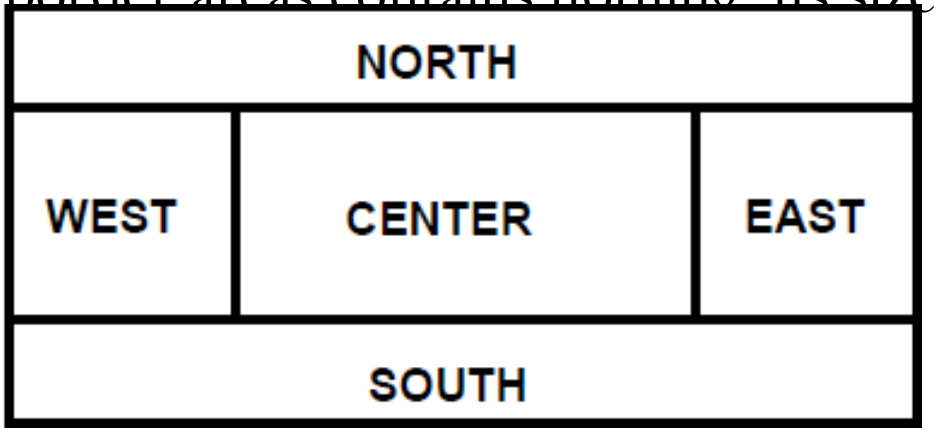
- Calling the pack () method for a window (JFrame) asks him to calculate his height (preferredSize), according to the components it contains, then to resize itself by adopting this size. This generally has the effect of making it as small as possible while respecting thepreferredSize of the components contained.

- The pack method automatically gives the window a minimum size while preserving the appearance of all its components and subcontainers.



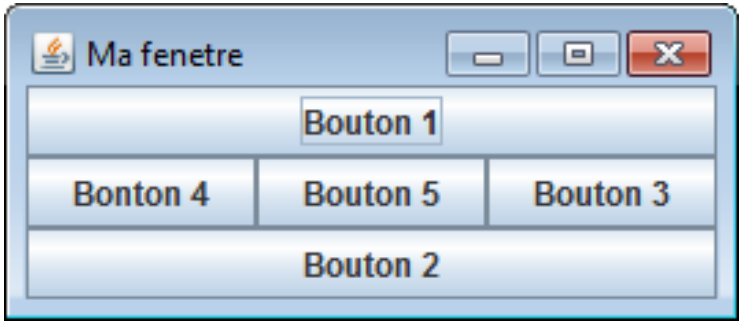
BorderLayout

- BorderLayout divides its workspace into five zones geographic : NORTH, SOUTH, EAST, WEST and CENTER.
- The components are added by name to these zones (only one component per zone).
- **Example:**
- add (new Button ("The button North ! "), BorderLayout.NORTH);
- If one of the border areas contains nothing, its size is 0.



BorderLayout

- Disposal strategy of BorderLayout
  - Components in NORTH and SOUTH adopt their preferred height and span the full width of the container.
  - The components located in EAST and WEST adopt their preferred width and extend vertically over the remaining space between the NORTH and SOUTH zones.
  - The component located in CENTER fills all the remaining space.



BorderLayout

- **BorderLayout** is a useful layout manager for large containers a user interface.

- By nesting a panel in each area of the container **BorderLayout**, then filling in each of these panels with other panels of various layouts, we can create rich user interfaces.

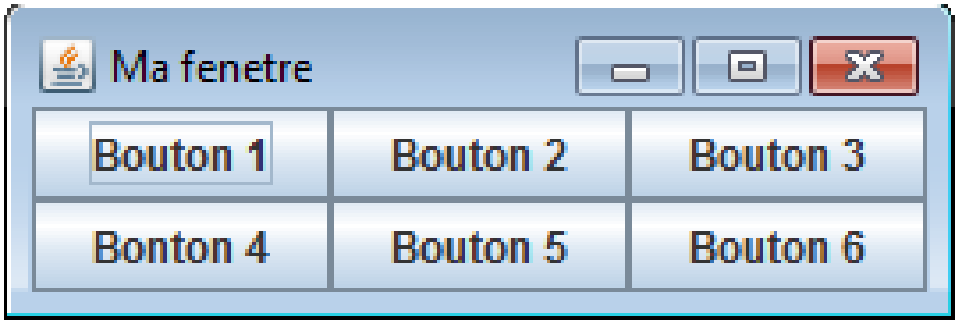
GridLayout

- **GridLayout** places the components on a grid of cells in rows and columns.
- It expands each component so that it fills the available space in the cell.
- All cells are exactly the same size and the grid is uniform.
- Following a resizing of the container, **GridLayout** changes the size of the cells so that they are as large as possible given the space available in the container.
- Construction of a **GridLayout** : *new GridLayout(3.2);*

GridLayout

- The number of rows or columns can be zero, but not both. At least one value must be specified so that the ***GridLayout*** can calculate the second.

- ***GridLayout*** is used to design a container where all components must be the same size, for example a numeric keypad or toolbar.



GridLayout

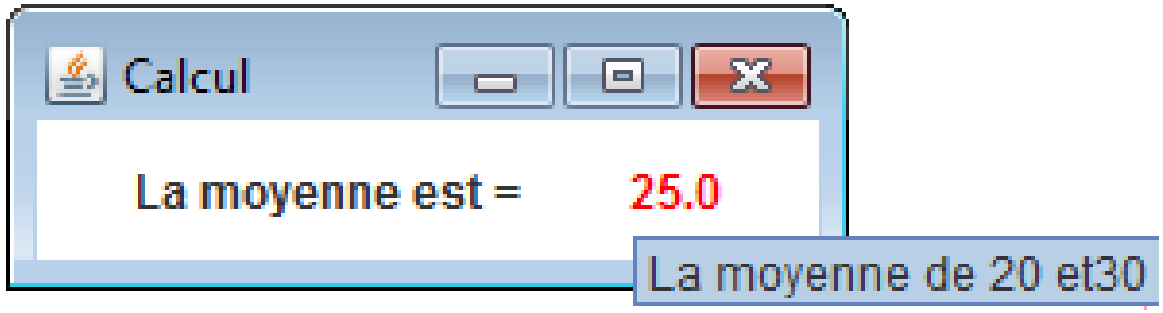
- The number of pixels between cells is specified using the methods *setHgap() and setVgap()*. Space horizontal and vertical space are zero by default.

- These two methods belong to the class Gridlayout;



Exercise

- *Write a JAVA program that calculates the average of 2 integers entered as parameters of the class, and display it on a window.*



Exercise

```
window2 package;
import javax.swing. *;
import java.awt. *;
public class Window2 extends JFrame{

 private JLabel l1, l2;

 public Window2 (String t, String a, String b)
 {
 super (t);
 setLocation(500,270);
 setSize(200.70);
 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

 l1 = new JLabel("The mean is =");
```

Exercise

```
JPanel p = (JPanel) this.getContentPane();
FlowLayout fl= new FlowLayout(FlowLayout.LEFT);
p.setLayout(fl);

double x = Double.parseDouble(a);
double y = Double.parseDouble(b);
Double res= new Double ((x + y) / 2);
l2 = new JLabel(res.toString());
String s1 = "the average of" + a + "and" + b;

l2.setToolTipText(s1);
l2.setForeground(Color.red);

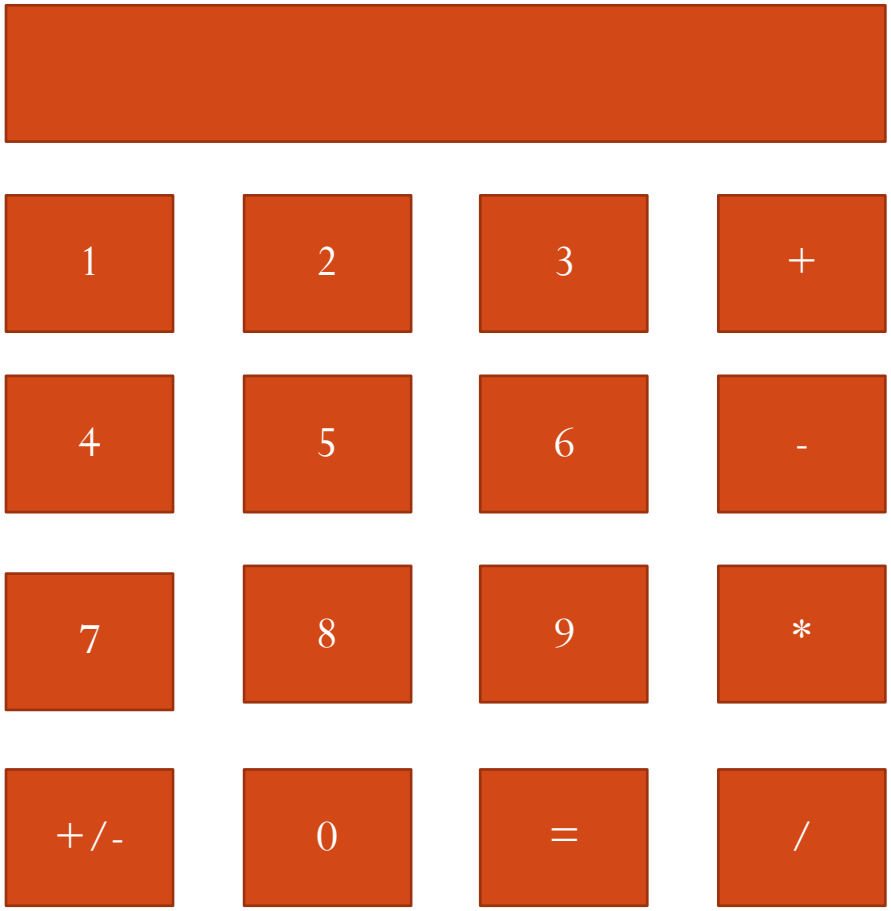
p.setBackground(Color.WHITE);

p.add (11, BorderLayout.CENTER);
p.add (12, BorderLayout.NORTH);
}
```

Exercise

```
public static void main (String [] args) {
Window2 f = new Window2 ("Window 2 ", args[0],args[1]);

 f.show();
 }
}
```


1
2
3
+
4
5
6
-
7
8
9
*
+/-
0
=
/

Event management

- When the user performs an action at the level of the graphical interface (mouse click, selection of an item, keystroke on the keyboard, etc), then an event is issued.
- An event is the instance of a class XXXEvent from the java.awt.event package.

- **Example:**
  - Following a mouse click in a window, it sends an event of type MouseEvent.
  - A button that receives a mouse click emits a class instance event ActionEvent.

Event management

- The component emitting the event is the s**ource of the event.** It transmits this event to one or more other objects, playing the role of**earphone** (**Listener).**

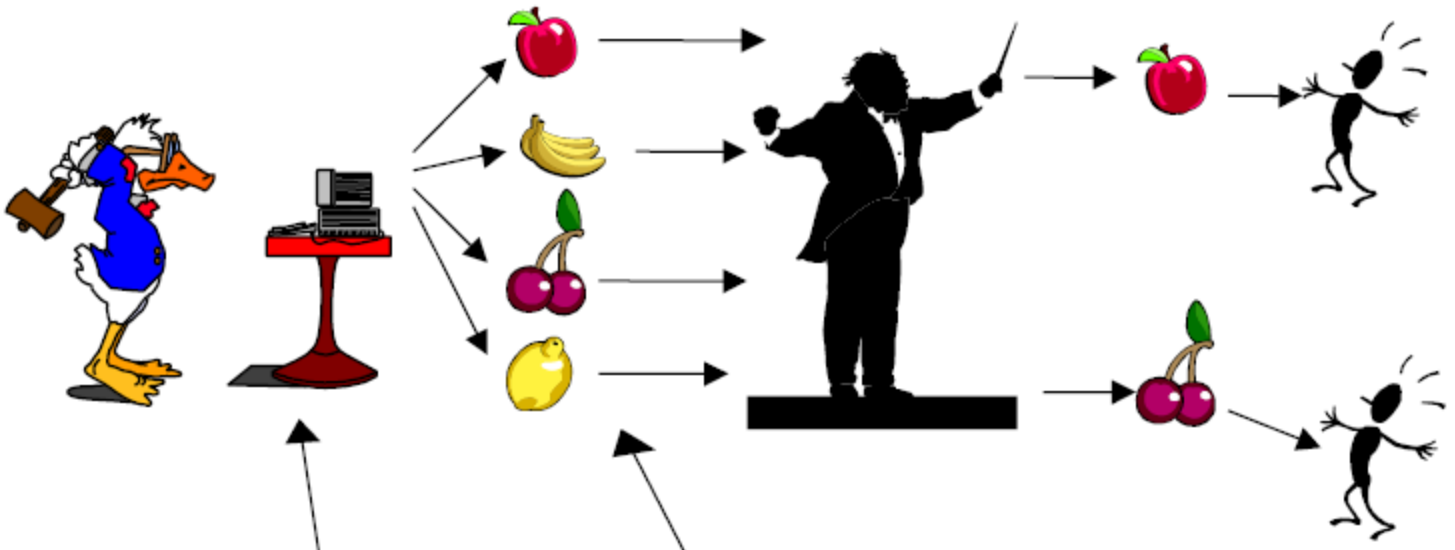
- Each component **SWING** is designed to be the source of one or more particular types of events.

- To process an event of type *XXXEvent*, a listener must implement the interface **XXXListener**

- Any listener of a component must register with this component using the method **addXXXListener().**

- A component can be its own listener.

Event management



L 'utilisateur réalise une action

Des événements sont alors générés

La Machine Virtuelle reçoit tous les événements

Seuls les événements écoutés sont transmis

Source de l 'événement

- JButton,
- JTextFiel
- ...

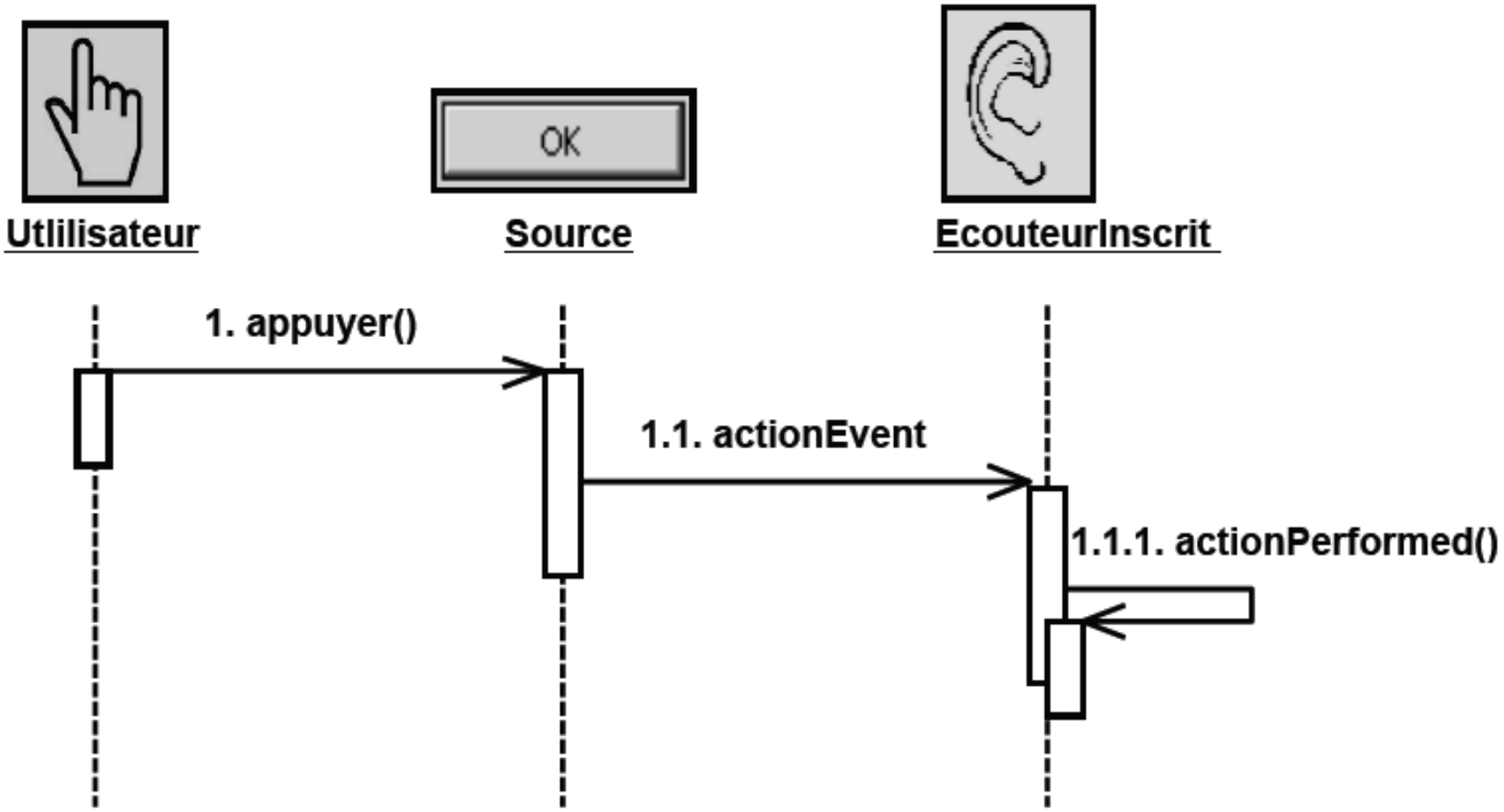
Événements

- ActionEvent
- MouseEvent
- ...

Écouteurs

- ActionListner
- MouseListner
- ...

Event management



Scénario de gestion de l'événement de clic sur un bouton.

Event management

- **Example1:**

```
import java.awt. *;
import java.awt.event. *;
import javax.swing. *;
public class Window6 extends JFrame
implements ActionListener {
private JButton b;
private JTextField t = new JTextFie
public Window6 () {
JPanel c =
(JPanel)this.getContentPane();
c.setLayout(new BorderLayout());
b = new JButton ("OK");
c.add (b, BorderLayout.SOUTH);
c.add (t, BorderLayout.NORTH);
b.addActionListener(this); }
```

Window6 is a listener for b. It must therefore implementActionListener

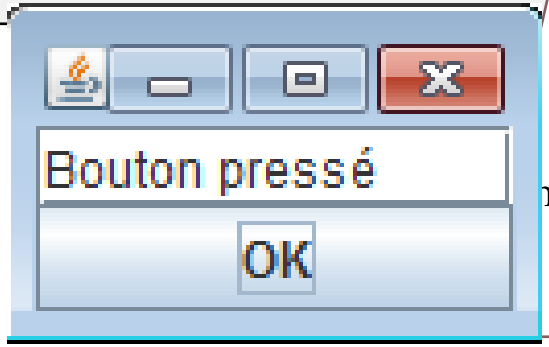
Window6 (this) registers as a listener of b.

Event management

- **Example1:**

```
import java.awt. *;
import java.awt.event. *;
import javax.swing. *;
public class Window6 extends JFrame
implements ActionListener{
private JButton b;
private JTextField t = new JTextField()
public Window6 () {
JPanel c = (JPanel)this.getContentPane
c.setLayout(new BorderLayout());
b = new JButton ("OK");
c.add (b, BorderLayout.SOUTH);
c.add (t, BorderLayout.NORTH);
b.addActionListener(this); }
```

```
public void actionPerformed (ActionEvent e)
{
 t.setText("Button pressed");
}
public static void main (String args [])
{
        Window6 f = new Window6 ();
        f.pack ();
        f.setVisible(true);
}
}
```



mplement

Event management

- **Example 2: Using inner classes**

- It's about a second solution to manage events.

- We create **inner classes** whose instances will only be used to listen to events.

- There is thus **one class per component** dedicated to the management of events that this component can generate.

Event management

```
package window;
import javax.swing. *;
import java.awt. *;
import java.awt.event. *;
public class Window extends JFrame {
private JButton b1;
private JTextField t;
 public Window ()
 { setLocation(250,250);
 setTitle("Window ");
  setSize(200,100);
 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE );
 JPanel p = (JPanel)this.getContentPane();
 setLayout(new GridLayout(2.1));
 b1 = new JButton("OK");
 t = new JTextField("Area of Text");
 b1.addActionListener(new OkListener());
 p.add (t);
 p.add (b1);
 }
```

```
class OkListener implements ActionListener{

 public void actionPerformed(ActionEvent e)
 {
 t.setText("OK button pressed");
 }
 }
```

Creating an instance of the inner class

Event categories

- Several types of events are defined in the package ***java.awt.event.***
- For each category of events, there is an interface that must be defined by any class wishing to listen to that category of events.
- This interface also requires that one or more methods be defined.
- These methods are called when events in this category occur.

Event categories

- **ActionListener**
  - Listen for a button click, carriage return in a text box, etc.
  - Requires the implementation of ***actionPerformed(ActionEvent e)***
- **TextListener**
  - Listen to the value change in a text box
  - Requires the implementation of ***textValueChanged(TextEvent e)***
- **ItemListener**
  - Listen to the selection of an item in a list
  - Requires the implementation of ***itemStateChanged(ItemEvent e)***
- **FocusListener**
  - Listen to the gain or loss of focus by a component.
  - Requires the implementation of ***focusGained(FocusEvent e) and focusLost(FocusEvent e)***

Event management

- The event object sent to listeners and passed as parameters of the corresponding functions can contain parameters of interest for the application.
  - For example, ***getX() and getY()*** on a ***MouseEvent*** return the coordinates of the mouse pointer position.
  - One of the information that is generally useful for processing an event is the source of that event. This information is obtained by calling the method***getSource()*** for this event.
  - The method *Object getSource()* is present in all event classes

Event management

```
import java.awt. *;
import java.awt.event
import javax.swing. *
public class Window7 extends JFrame
implements ActionListener{
private JButton b1 = new JButton ("OK");
private JButton b2 = new JButton ("To leave");
private JTextField t = new JTextField();
public Window7 () {
Container c =this.getContentPane();
c.setLayout(new BorderLayout());
c.add (b1, BorderLayout.CENTER);
c.add (b2, BorderLayout.SOUTH);
c.add (t, BorderLayout.NORTH);
b1.addActionListener(this);
b2.addActionListener(this);
}
```

```
public void actionPerformed(ActionEvent e)
{
if ((JButton) e.getSource() == b1)

 t.setText("OK button pressed");
else
 t.setText("EXIT button
 hurry");
}

public static void main (String args [])
{
Window7 f = new Window7 ();
f.pack ();
f.setVisible(true);
}
}
```

e.getSource() returns the source object of the event

The 2 buttons have the same earpiece (the window)

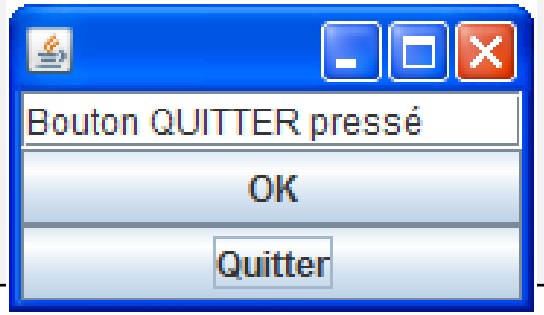
Event management

```
import java.awt. *;
import java.awt.event. *;
import javax.swing. *;
public class Window7 extends JFrame
implements ActionListener{
private JButton b1 = new JButton ("OK");
private JButton b2 = new JButton ("To leave");
private JTextField t = new JTextField();
public Window7 () {
Container c =this.getContentPane();
c.setLayout(new BorderLayout());
c.add (b1, BorderLayout.CENTER);
c.add (b2, BorderLayout.SOUTH);
c.add (t, BorderLayout.NORTH);
b1.addActionListener(this);
b2.addActionListener(this);
}
```

```
public void actionPerformed(ActionEvent e)
{
if ((JButton) e.getSource() == b1)

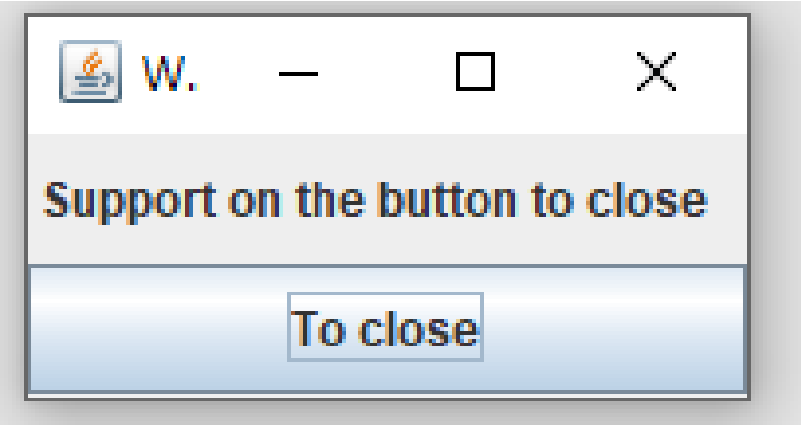
 t.setText("OK button pressed");
else
 t.setText("EXIT button
 hurry");
}

public static void main (String args [])
{
Window7 f = new Window7 ();
f.pack ();
f.setVisible(true);
}
}
```



Exercise

- Write a program that displays a window with a button. When you click on the button, you close the window and the program stops.



Solution

```
import javax.swing. *;
import java.awt. *;
import java.awt.event. *;
public class Window extends JFrame implements ActionListener{
private JLabel l1;
private JButton b;
public Window ()
 { setLocation(250,250);
 setTitle("Window ");
 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE );
 JPanel p = (JPanel)this.getContentPane();
 p.setLayout(new GridLayout(2,1));
 b = new JButton("To close");
 l1 = new JLabel(" Support on the button to close ");
 b.addActionListener(this);
 add(l1);
 add(b);
 }
 public void actionPerformed(ActionEvent e)
 { System.exit(0); }
```

```
 public static void main (String [] args) {
Window fen = new Window();
 fen.setVisible(true);
 fen.pack();
 }
}
```

Exercise

- Write a program that displays a window on which we see displayed the number of clicks made on the window.



Exercise

- **Indication:**

- Click on button, move the pointer.

- Interface methods MouseListener

  - **public void mouseClicked(MouseEvent e)**
  - **public void mouseEntered(MouseEvent e)**
  - **public void mouseExited(MouseEvent e)**
  - **public void mousePressed(MouseEvent e)**
  - **public void mouseReleased(MouseEvent e)**

- Events generated by *Component*

- Recording method **addMouseListener**

Solution

```
public class Window extends JFrame implements MouseListener{
private JLabel l1, l2;
private static int nb = 0;
 public Window ()
 { setLocation(250,250);
 setTitle("Counter");
 setSize(300.70);
 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE );
 JPanel p = (JPanel)this.getContentPane();
 setLayout(new FlowLayout(FlowLayout.LEFT));

 l1 = new JLabel("Number of mouse clicks:");
 l2 = new JLabel("0");
 add(l1);
 add(l2);

 addMouseListener(this);
 }
```

Solution

```
public void mouseClicked(MouseEvent e)
 {
 String s = "";
 s + = ++ nb;
 l2.setText(s);

 }

// Insert prototypes without defining their bodies.

 public void mouseEntered(MouseEvent e) {};
 public void mouseExited(MouseEvent e) {};
 public void mousePressed(MouseEvent e) {};
 public void mouseReleased(MouseEvent e) {};
```

Adapters

- Reminder: a class which implements an interface must redefine all the methods of the interface

- However, some interfaces `Listener` have a lot of methods, which forces you to redefine methods that you have no use for

- To remedy this Java offers Adapter classes for certain interfaces `Listener`

Adapters

- Instead of implementing the interface, it will suffice to inherit from the Adapter class.
- In this case the developer redefines only the methods which interests him.
- Java thus offers a set of Adapter class
- The naming principle is simple: a named class **XXXAdapter** corresponds to the earphone interface **XXXListener**

Adapters

- WindowAdapter
- MouseAdapter
- MouseMotionAdapter
- KeyAdapter
- FocusAdapter
- ComponentAdapter
- ContainerAdapter

Exercise

- **Solution 2: Create an inner class that inherits from the class `MouseAdapter`**

```
public class Window extends JFrame{
private JLabel l1, l2;
private static int nb = 0;
 public Window ()
 { setLocation(250,250);
 setTitle("Counter");
 setSize(300.70);
 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
 JPanel p = (JPanel)this.getContent
 setLayout(new FlowLayout(FlowLa

 l1 = new JLabel("Number of mous
 l2 = new JLabel("0");
 add(l1);
 add(l2);

 addMouseListener(new ListenMouse());
 }
```

```
public class ListenMouse extends MouseAdapter{
 public void mouseClicked(MouseEvent e)
 {
 String s = "";
 s + = ++nb;
 l2.setText (s);

 }
```

Exercise

- **Solution 2: Create an anonymous class that inherits from `MouseAdapter`**

```
…… ..
l1 = new JLabel("Number of mouse clicks:");
 l2 = new JLabel("0");
 add(l1);
 add(l2);

 addMouseListener(new MouseAdapter() {public void
mouseClicked(MouseEvent e)
 {
 String s = " ";
 s + = ++ nb;
 l2.setText(s);

 }
}
 );
```

Dialogs

- A dialog is a container. It allows you to group any components in a kind of window that you make appear or disappear.

- Java provides a number of standard dialog boxes obtained using methods of the class ***JOptionPane*** : message boxes, confirmation boxes, input boxes and option boxes.
  - Class **JDialog** allows you to build custom dialog boxes.

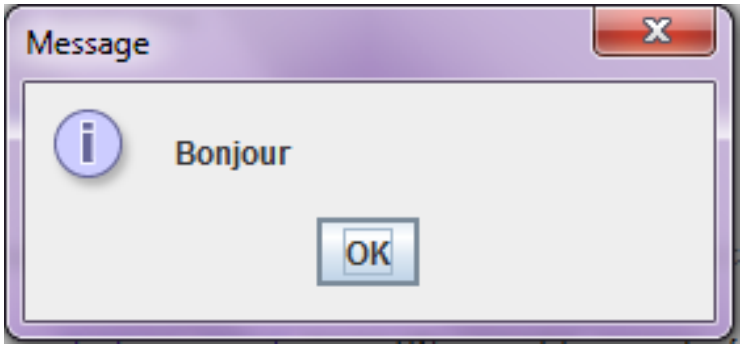
Dialogs

- Message boxes
  - A message box provides the user with a message that remains displayed until the user acts on the OK button. It is built using the class method

    static void showMessageDialog (parent, message)

  of style ***JOptionPane.***
  - Example

    JOptionPane.showMessageDialog (null , "Hello") ;}}



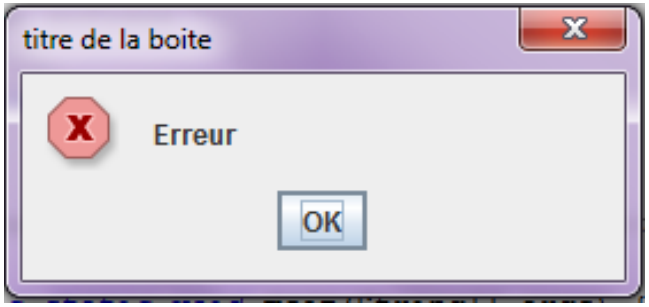
Dialogs

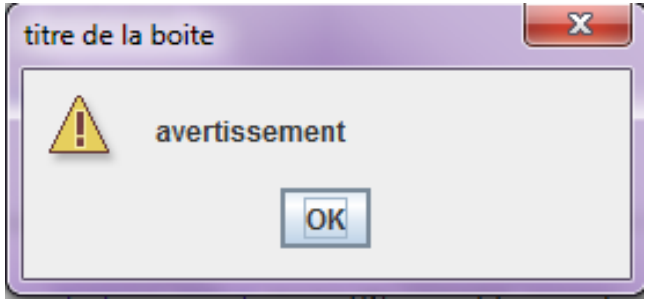
- There is a variant of the method ***showMessageDialog()*** which allows you to choose the title of the box and the type of icon among the elements of the following table (the parameters are integer constants of the class ***JOptionPane).***

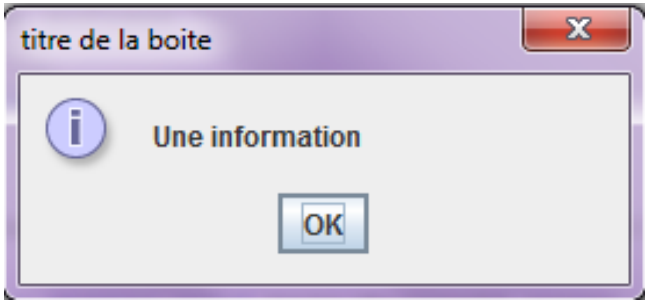
Setting	Icon type
static int JOptionPane.ERROR_MESSAGE	Fault 
static int JOptionPane.INFORMATION_MESSAGE	Information 
static int JOptionPane.WARNING_MESSAGE	Warning 
static int JOptionPane.QUESTION_MESSAGE	Question 
static int JOptionPane.PLAIN_MESSAGE	No icon

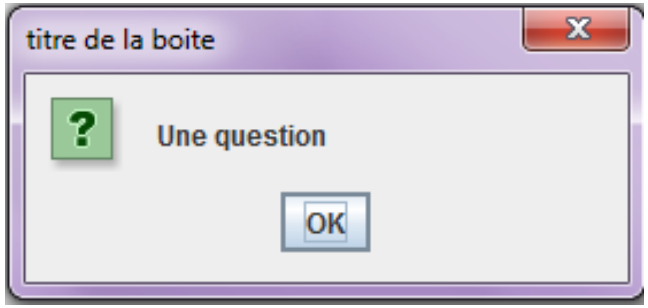
Dialogs

- Example
  - *JOptionPane.showMessageDialog(parentWindow, content, title,type_icone);*
  - *JOptionPane.showMessageDialog (null, "warning", "box title", JOptionPane. WARNING_MESSAGE);*





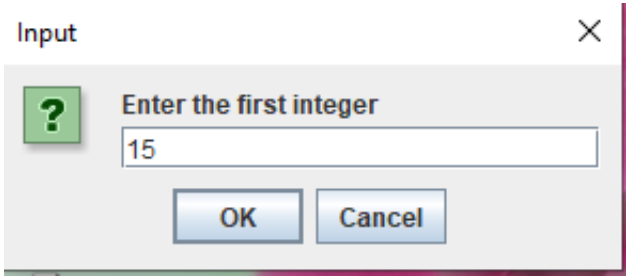




Dialogs

- Entry boxes
  - An input box allows the user to provide information in the form of a character string.
    - She is constructed using the class method

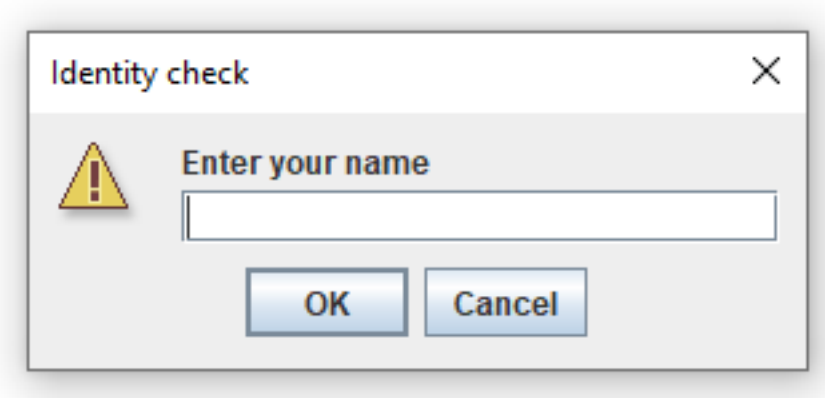
    Static String showInputDialog (parent, message) of style ***JOptionPane.***
  - Example
    - *JOptionPane.showInputDialog (null , "Give a text");*



- The return value of the method ***showInputDialog is either a*** type object **String** containing the text provided by the user, i.e. the value null if the user has not confirmed their entry with the OK button.

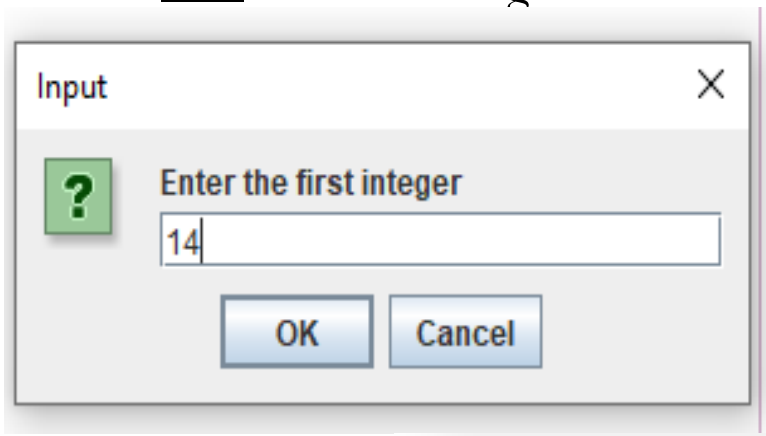
Dialogs

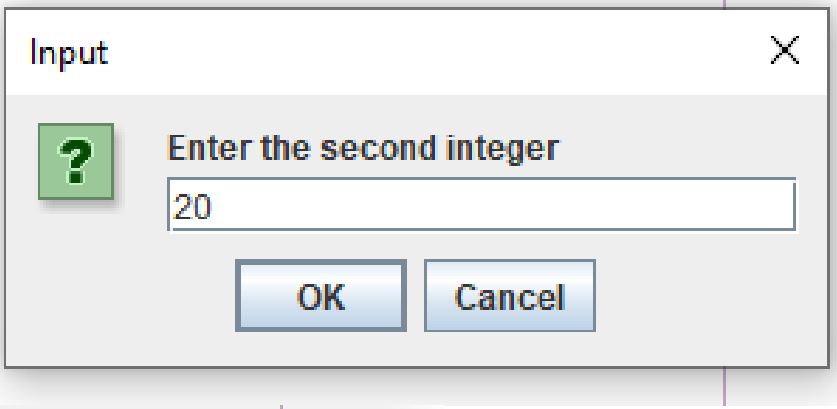
- There is a variant of the method ***showInputDialog who*** also allows you to choose the title of the box and the type of icon.
- **Example**
  - *JOptionPane.showInputDialog (null, "Enter your name", "Identity check", JOptionPane.WARNING_MESSAGE);*

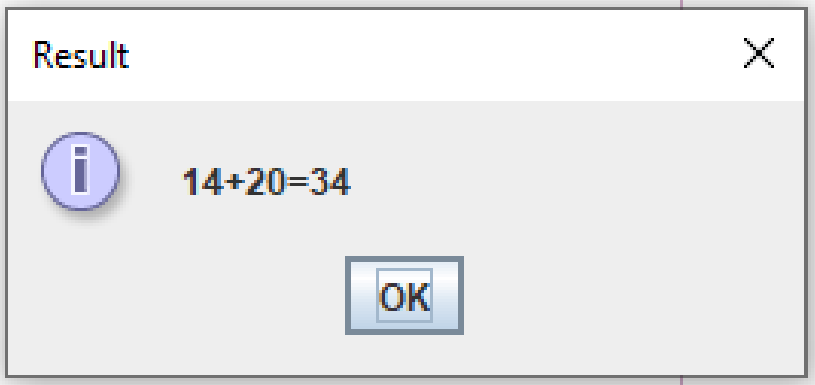


Dialogs

- **Exercise:**
  - Write a program allowing to ask the user to give two integers and to display their sum thereafter.
    - **NB:** Use the dialogue boxes to answer the exercise.





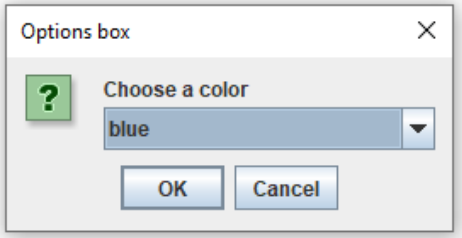


Dialogs

- The option boxes:
  - An options box allows the user to choose a value from a list of values, via a combo box. It is built using the class method
    - ***static Object showInputDialog (arg1, arg2, arg3, arg4, arg5, arg6, arg7)***

  of style ***JOptionPane.***

  - The return value of the method ***showInputDialog*** *is either a* Object containing the value selected by the user, or **null** if the user did not confirm his entry using the button **OK.**



  - Example

```
String [] colors = {"red", "blue", "gray", "green", "yellow", "black"};
JOptionPane.showInputDialog(null, "Choose a color", "Options box",
    JOptionPane.QUESTION_MESSAGE, null, colors, colors [1]);
```

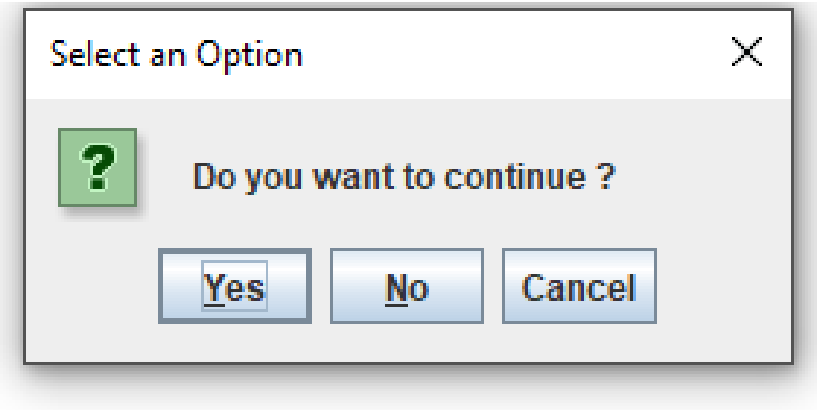
Dialogs

- Confirmation boxes
  - A confirmation box offers the user a choice of yes / no / cancel.
  - It is built using the class method:

static int showConfirmDialog (parentWindow, message)

of style JOptionPane.

JOptionPane.showConfirmDialog (null, "Do you want to continue ?");



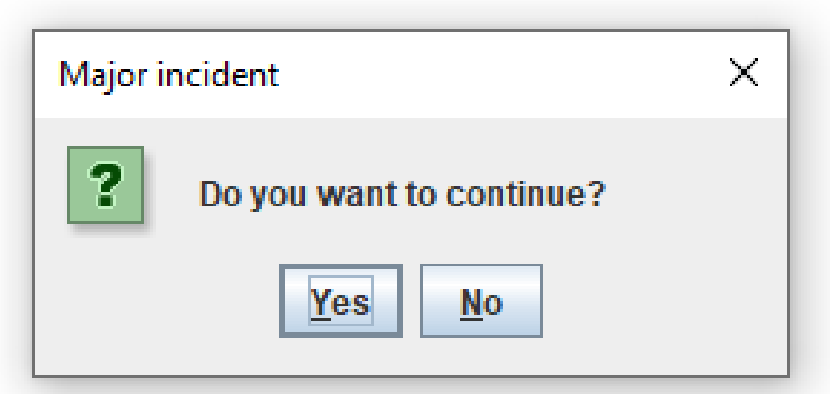
Dialogs

- There is a variant of the method ***showConfirmDialog() who*** allows you to choose the title of the box and the nature of the buttons found there among the elements of the following table (the parameters are integer constants of the class ***JOptionPane).***

Setting	Box type
JOptionPane.DEFAULT_OPTION	Fault
JOptionPane.YES_NO_OPTION	buttonsYES and NO
JOptionPane.YES_NO_CANCEL_OPTION	YES, NO and CANCEL buttons
JOptionPane.OK_CANCEL_OPTION	OK and CANCEL buttons

Dialogs

- Example
  - *int n = JOptionPane.showConfirmDialog (null, "Do you want to continue?", "Major incident", JOptionPane.YES_NO_OPTION);*



  - The return value of the method showConfirmDialog specifies the action taken by the user as an integer having one of the following constants:
    - JOptionPane.YES_OPTION (0),
    - JOptionPane.OK_OPTION (0),
    - JOptionPane.NO_OPTION (1),
    - JOptionPane.CANCEL_OPTION (2),
    - JOptionPane.CLOSED_OPTION (-1).

Qassim University جامعة القصيم

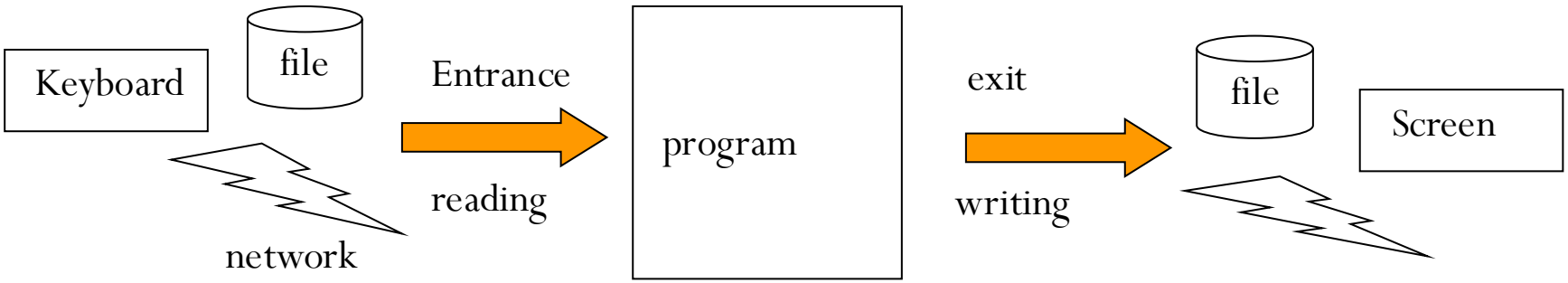
Chapter 3: Java inputs and outputs

Dr Hedia ZARDI

H.zardi@qu.edu.sa

What is I / O?

- A program often needs to read data from an external source or write data to an external recipient.
- The source and destination of these exchanges can be of multiple types: keyboard, screen, file, network socket, etc.
- Likewise, the nature of the data exchanged can be diverse: text, images, sound, etc.



Flows

- In java, the flow mechanism ('stream') is used to represent all these data sending / receiving processes.
- Flows always process data in a way **sequential** (see sequential files in progressalgo):
  - when the flow opens, the current position is defined (e.g .: at **start** of a file open for reading);
  - as soon as a data is read or written, the current position **advance**; the stream has no memory and always gives access to a single data at a given time.

```
flow opening
as long as there is data
 read / write data in the stream
end while
flow closure
```

Stream categories

- 2 main distinctions:
  - the input streams (read) and the output streams (write),
  - character streams (texts, html, xml…) And byte streams (whole / real numbers, images, sounds…).
- Java.io package: all flow management classes
- Class names are broken down into a prefix followed by a suffix.
- Suffixes (abstract stream classes):

	Character stream	*Byte stream*
Input stream	***Reader***	***InputStream***
Output flow	***Writer***	***OutputStream***

- Prefixes: indicate **what the flow communicates with** (***File, Object, String***) or the type of treatment (**filtered**) associate (***Buffered, Data***…).

The different filters

Type of treatment	*description*	*prefix*	*entrance*	*output*
buffering	the flow data passes through a **buffer** (buffer) to optimize reads / writes	**Buffered**	Yes	Yes
data conversion	processing bytes as **typed data (double, int, ...)**	**Data**	yes for byte streams	yes for byte streams
impression	formatted print	**Print**	no	Yes
serialization	processing bytes as**objects**	**Object**	yes for byte streams	yes for byte streams
bytes / characters conversion	to convert bytes to characters (byte stream / character stream gateway)	**InputStream / OutputStream**	yes for character streams	yes for character streams

The main classes of java.io

- We thus obtain a large number of classes:
  - *FileReader* (character file read),
  - *FileWriter* (write character file),
  - *BufferedReader* (character file with read buffer),
  - *BufferedWriter* (character file with write buffer),
  - *FileInputStream* (file bytes read),
  - *FileOutputStream* (file bytes in writing),
  - *ObjectInputStream* (read object file),
  - *ObjectOutputStream* (write object file), etc.

Character flows

- Carry data in the form of characters (unicode) on 2 bytes.
- The classes that manage character flows inherit from one of the two abstract classes *Reader* Where *Writer*.

Reader

- **The abstract class *Reader*** = character stream read:
  - *int read()* : returns the character read (integer whose value is the code unicode read character) or -1 if the end of the flow is reached.
  - *int read(char[ ])* : read several characters and place them in a character array, return the number of characters read or -1 if the end of the stream is reached.
  - *void close ()* : closes the flow and frees the associated resources.
  - These methods can raise *IOException*.

Writer

- **The abstract class *Writer*** = stream of characters in writing:
  - *void write(int)* : write the character as a parameter (integer interpreted as unicode) in the stream.
  - *void write(String)* : write the character string as a parameter in the stream.
  - *void close ()* : closes the flow and frees the resources.

  - These methods can raise *IOException*.

FileReader

- **Class *FileReader*** = stream of characters being read on a **file**.

  - Several manufacturers who can lift a *FileNotFoundException* (derived from *IOException*). The most common opens a text file for reading from its name.

```
FileReader file = new FileReader("myfile.txt");
```

  - *read* and *close* are inherited of *Reader*.

FileWriter

- **Class *FileWriter*** = stream of characters being written to a file.
  - Several manufacturers who can lift a *FileNotFoundException* :

FileWriter(String) : if the file name does not exist so the file is created; otherwise, the stream is open for writing to this file (including data is overwritten).

FileWriter(String, boolean) : the boolean is used to specify whether the data is added at the end of the file (value true) or overwrite existing data (false).

```
FileWriter file = new FileWriter ("myfile.res");
```

  - *write* and *close* are inherited from *Writer*.

The character streams " buffered "

- Data buffering to process a set of characters representing a **line** rather than read / write **character by character** (performance improvement).

BufferdReader

- **Class *BufferedReader*** = buffered character stream read :
  - Builder:
    - *BufferedReader(Reader)*: the parameter provided must correspond to the stream to read. Subtyping is used to pass as a parameter the reference to an object of any subclass of*Reader*.

      BufferedReader file = new BufferedReader(new FileReader ("myfile.txt"));

    - In addition to the methods *read* inherited from the class *Reader, BufferedReader* offers a line-by-line reading method that can raise a *IOException* : *String readLine()* : read the next line of characters in the stream.
      - A line is a sequence of characters that ends with a line break '\ n'. Makes **null** at the end of the file. Otherwise, make the line without the end character.

  - *close* inherited from *Reader*.

Exercise

- Write a program that displays the contents of a text file on the screen.

Solution

```
import java.io. *;
public class TestBufferedReader {
 private Source string;

 public TestBufferedReader(String src) {
 this.source = src;
 }

 public void reading( ) {
 try {
 String line;
 BufferedReader file = new BufferedReader(new FileReader(source));

 while ((line = file.readLine())! = null)
 {
 //reading and test (practice!)
  System.out.println (line);
 }
 file.close();
 } catch (IOException e) {
 e.printStackTrace();
 }
 }
 public static void main (String [] args) {
 TestBufferedReader br = new TestBufferedReader("source.txt"); reading();
 }
}
```

BufferedWriter

- **The class *BufferedWriter*** = write buffered character stream:
  - Builder
    - *BufferedWriter(Writer)*: the parameter corresponds to the stream to write. Subtyping is used to pass as a parameter the reference to an object of any subclass of *Writer*.

      BufferedWriter file = new BufferedWriter(new FileWriter ("myfile.txt"));

    - In addition to the methods *write* inherited from *Writer*, *BufferedWriter* redefines writing methods in order to optimize them.
      - The method *newLine()* allows you to write a line separator and it can raise a *IOException*.

    - *close* inherited from *Writer*.

Exercise

- Write a program that writes to a text file.

Solution

```
import java.io. *;
public class TestBufferedWriter {
 private String dest;
 public TestBufferedWriter(String d) {
 this.dest = d;
 }
 public void processing( ) {
 try {
 int number = 12345;
 BufferedWriter file = new BufferedWriter(new FileWriter(dest));
 file.write("Hello");
 file.newLine();
 file.write("The number is");
 file.write(":" + number);
 file.close();
 } catch (IOException e) {
 e.printStackTrace();
 }
 }
 public static void main (String [] args) {
 TestBufferedWriter bw = new TestBufferedWriter("print.txt");
 bw.processing( );
 }
}
```

PrintWriter

- **Class *PrintWriter*** = character stream to write formatted data (integers, reals, objects …)
  - Several manufacturers including:
    - *PrintWriter(Writer)* : the parameter provided specifies the stream in which the data is written.

      PrintWriter file = new PrintWriter(new FileWriter ("myfile"));
  - In addition to the methods *write* inherited from *Writer*, the class defines methods for formatted data:
    - *void print(boolean)* : write a boolean (true or false).
    - *void print(double)* : write a real (string of character).
    - *void print(int)* : write an integer (string of character).
    - *void print(Object)* : write an object (via a call to the method *()* of this object).
    - *void print(String)* : write a string.

PrintWriter

- *void println()* : write a line separator.
- *void println(boolean)* : write a boolean then a line separator.
- *void println(double)* : write a real then a line separator.
- *void println(int)* : write an integer then a line separator.
- *void println(Object)* : write an object then a line separator.
- *void println(String)* : write a character string then a line separator.
- *void printf(String format, Object … args)* : writes a character string formatted from a variable number of objects (notation *Object … args*) (as in the C language).

- *close* inherited from *Writer*.

Example

```
float To = 2565.634;
float b = 20.0;
int c = 123;

PrintWriter file = new PrintWriter(new FileWriter("file.txt"));
file.println(a + "*" + b + "+" + c + "=" + (a * b + c));
file.printf("% f *% f +% d =% f \ n", a, b, c, a * b + c);

//format objects
//% f: real% d: integer% n: separator of line

file.printf("% 3.2f *% 3.2f +% x =% 3.2f \ n", a, b, c, a * b + c);

//%3.2f: real with 2 digits after the decimal point and at least 3
  digits
//% x: integer in hexadecimal
```

Written in the file.txt file:

2565.634 * 20.0 + 123 = 51435.68

2565.634000 * 20.000000 + 123 = 51435.680000

2565.63 * 20.00 + 7b = 51435.68

Byte streams

- Carry binary data in the form of bytes. They can process any type of data.

- The classes that manage the byte streams inherit from one of the abstract classes *InputStream* Where *OutputStream*.

InputStream

- **The abstract class *InputStream*** = stream of read bytes:
  - *int read()* : returns the read byte (integer between 0 and 255) or -1 if the end of the stream is reached.
  - *int read(byte[])*: reads several bytes and places them in a byte array. Returns the number of bytes read or -1 if the end of the stream is reached.
    - NB: *byte* is a single-byte encoded integer type.
  - *void close ()* : closes the flow and frees the associated resources.
- These methods can raise *IOException*.

OutputStream

- **The abstract class *OutputStream*** = stream of write bytes:
  - *void write(int)* : write the byte as a parameter (least significant part of the int) in the stream.
  - *void write(byte [])* : write the byte array to the stream.
  - *void close ()* : closes the flow and frees the resources.
- These methods can raise *IOException*.

FileInputStream

- **Class *FileInputStream*** = stream of bytes read from a file:

  - Several manufacturers who can lift a *FileNotFoundException*. The most common opens a byte file for reading from its name.

```
FileInputStream file = new FileInputStream ("myfile.dat");
```

  - *read* and *close* are inherited of *InputStream*.

FileOutputStream

- **Class *FileOutputStream*** = stream of bytes in writing to a file:

  - Several manufacturers who can lift a *FileNotFoundException*, specifically :

FileOutputStream(String) : if the file name does not exist so the file is created; otherwise, the stream is open for writing to this file (including data is overwritten).

```
FileOutputStream file = new FileOutputStream ("myfile.dat");
```

  - *write* and *close* are inherited from *OutputStream*.

Exercise

- Write a program that copies the contents of one file to another file, the copy should be byte by byte.

Solution

```
import java.io. *;
public class CopyFile {
 private String source, destination;
 public CopyFile(String src, String dest) {
 this.source = src; this.destination = dest;
 }
 public static void main (String [] args) {
 CopyFile cf = new CopyFile("source "," copy");
 cf.copy ();
 }
 public void copy () {
 try {
 FileInputStream fis = new FileInputStream(source);
 FileOutputStream fos = new FileOutputStream(destination);
 int oc =fis.read();
 while(oc! = -1) {
 fos.write(oc);
 oc =fis.read();
 }
 fis.close(); fos.close();
 } catch (IOException e) {
 e.printStackTrace();
 }
 }
}
```

Binary data streams

- Reading a stream of bytes byte by byte is not always practical. The classes ***DataInputStream*** and ***DataOutputStream*** allow to directly read / write full, real, Boolean typed data …

DataInputStream

- **Class *DataInputStream*** = typed data stream for reading:
  - Manufacturer: DataInputStream(InputStream): creates a stream of typed data for reading from the stream of bytes in parameter.
    - *DataInputStream file = new DataInputStream(new FileInputStream("myfile.dat"));*
  - In addition to the methods inherited from InputStream, the class DataInputStream defines methods for reading typed data:
    - *boolean readBoolean()* : reads a boolean (one byte).
    - *int readInt()* : read an integer (4 bytes).
    - *double readDouble()* : reads a real (8 bytes).

    These methods can raise ***EOFException* if the end of flow has been reached** or some *IOException* if an I / O error has occurred.
  - *close ()* is inherited from *InputStream*.

DataOutputStream

- **Class *DataOutputStream*** = data stream typed for writing:
  - Builder : *DataOutputStream(OutputStream)* : creates a stream of typed data for writing from the stream of bytes in parameter.

    DataOutputStream file = new DataOutputStream (new FileOutputStream("myfile.out"));

  - In addition to the methods *write* inherited from the class *Output Stream*, the class *DataOutputStream* defines several methods for writing typed data:
    - *void writeBoolean(boolean)* : write the boolean in parameter.
    - *void writeInt(int)* : write the integer as a parameter.
    - *void writeDouble(double)* : write the real in parameter.

These methods can raise *IOException*.

- *close ()* is inherited from *OutputStream*.

Object streams - serialization

- The **serialization** allow write (then read) objects in byte streams. Java transforms the object **and all the objects related to it** into a sequence of bytes which can in turn be reinterpreted as the original object.

- Serialization allows an object to be easily saved to a file or transferred over the network, and the deserialization is the reverse process.

- It is up to the programmer to check that the objects he manipulates are serializable (i.e. the transformation can be applied to them).
  - The programmer indicates this by specifying that the class implements **predefined interface *Serializable*.**

- An object is serializable if all its instance attributes are :
  - either of a **base type** from Java (*int, double*…).
  - either of a **object type serializable**.

Example

```
public class A {
 private int x; // A is serializable
}
public class B {
 private Double v; // B is serializable
 private A ref;
}
```

To allow Java to serialize (transform into bytes) objects from A and B you have to write:

```
public class A implements Serializable {
 private int x;
}
public class B implements Serializable {
 private Double v;
 private A ref;
}
```

Object streams - serialization

- We can read and write objects serializable in byte streams thanks to the classes:
  - *ObjectInputStream*
  - *ObjectOutputStream*.

Example: writing a point and re-reading

```
import java.io. *;
public class Point implements Serializable {
 private int abscissa, ordinate;
 public Point (int x, int y) {
 abscissa = x;
 ordered= y;
 }
}
```

```
public class TestObjectStreams {
 private Point point;
 private String name; // file name
 public TestObjectStreams(Point pt, String nf) {
 this.point = pt;
 this.name = nf;
 }
 public static void main (String [] args) {
 TestObjectStreams tos = new TestObjectStreams(new Point
(20.30), "ficobj");
 tos.save();
 tos.charge();
 }
```

Example: writing a point and re-reading

```
public void Save() {
 try {
 ObjectOutputStream oos= new ObjectOutputStream(new
   FileOutputStream(name));
 oos.writeObject(point);
 oos.close();
 } catch (IOException e) {
 System.out.println ("I / O error");
 }
}
```

Example: writing a point and re-reading

```
    public void charge() {
     try {
     ObjectInputStream ow= new ObjectInputStream(new
       FileInputStream(name));
     point = (Point) (ow.readObject());
     ow.close();
     }
     catch (IOException e)
     {
     System.out.println ("I / O error");
     }
     catch (ClassNotFoundException e)
     {
     System.out.println ("error out of I / O");
     }
    }

}
```

The Scanner class

- Facilitates reading in a stream (package **java.util**). Particularly useful for reading data from the keyboard (*System.in*) in a console-type application.

System.in is a **InputStream**

System.out is a **PrintStream**

The method ***next()*** blocks execution until data is read and returns it as a string.

Example:

```
public class TestScanner {
 public static void main (String [] args) {
 To scan to scan = new Scanner (System.in);
 String s = scanner.next();
 scanner.close();
 }
 }
```

The Scanner class

- ***nextInt(), nextDouble(), ...:***
- The methods nextTYPE() where TYPE represents a base type block execution until data is read, attempt to convert it to type TYPE and throw an exception ***InputMismatchException*** if the read data cannot be converted.
- ***hasNextInt(), hasNextDouble(), hasNextBoolean(),...:***
- the methods hasNextTYPE() return a boolean that indicates if the data can be interpreted as being of type TYPE.
- **Example :**

```
System.out.print ("Next integer =");
 if (scanner.hasNextInt()) {
 n = scanner.nextInt();
 } else {
 System.out.println ("Type wrong: "+ scanner.next());
 }
```

Exercise 1 : Part 1

- Write a program that asks the user to enter a set of persons by giving the last name, first name and year of birth, then the three pieces of information will be added at the file "People.txt" with the following format: last_name; first name; year_of_Birth.
- After adding a new person, the program asks the user if he wants to continue adding other persons or not using this message:

"Do you want to add another person (y/n)?"

- If the user press Y, so he can add a new person. If not the execution of the program ends.

Part 2

- Write a program that reads the data saved in the people.txt file line by line. For each line we construct a Person type object from the 3 pieces of information found in this line (which are separated by a ";", use the split method of the String class), this object will then be saved in the file people_objects.txt as an object.

Part 3

- Write a program that reads an object stream from the people_objets.txt file (part 2) and display the read informations.

Chapter 4
Access to databases

Dr Hedia ZARDI

H.zardi@qu.edu.sa

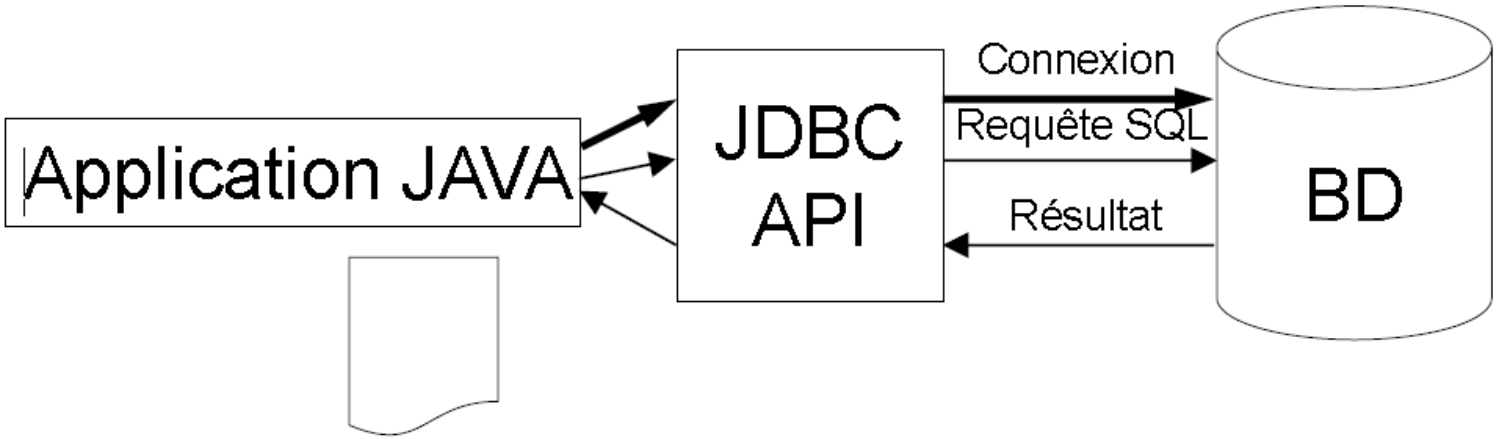
JDBC: Java DataBase Connectivity

- The JDBC API contains a series of interfaces designed to allow the developer of applications, which work on databases, to do so regardless of the type of database used.

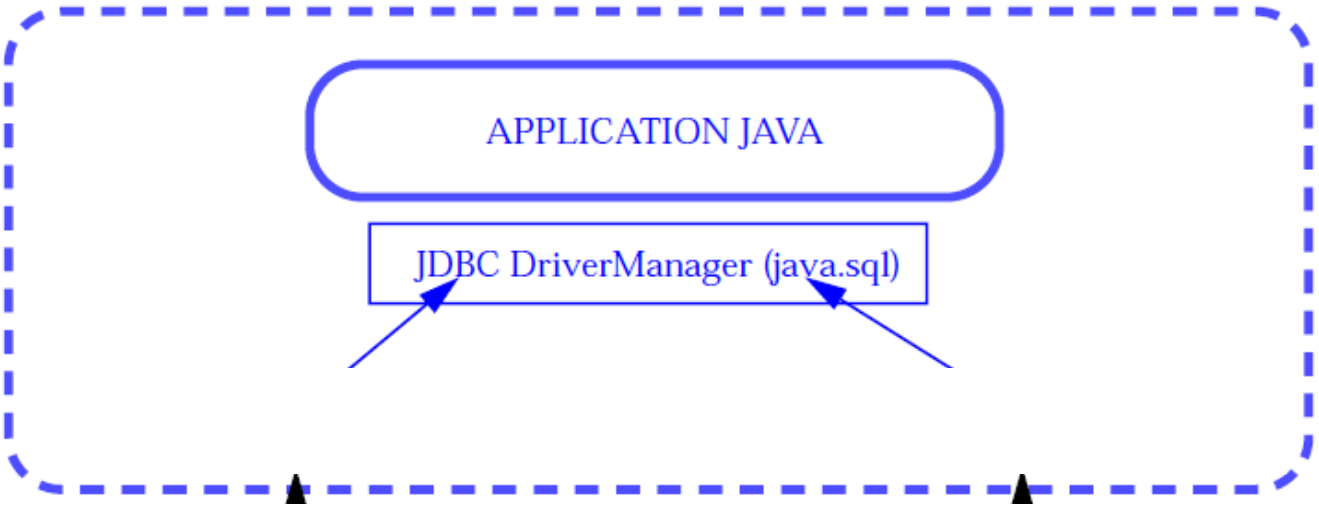
- Packages:
  - API package: java.sql. *
  - DBMS driver package: for each DBMS there is a free archive / library / library (.jar) and contains the driver classes.

Role of the JDBC API

- Create the connection to the database
- Send SQL statements
- Exploit the results



Role of the JDBC API



ODBC: Open DataBase Connectivity

- ODBC:
  - is a middleware (**middleware**) which allows a computer application, by a single process, to handle several databases which are made available by database management systems (DBMS) each having its own process.
  - This software was implemented in 1992 by Microsoft for operating systems **Windows**, then later by other editors for other operating systems such as **Unix** and **the Java platform**.
  - Requires a configuration phase

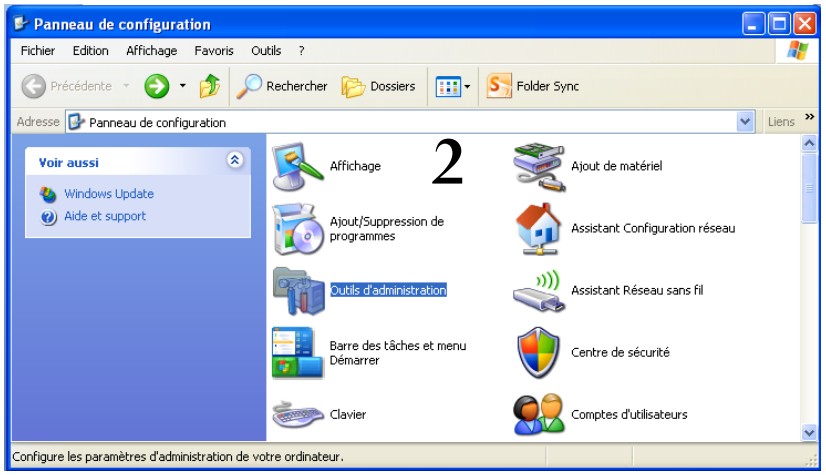
ODBC: example

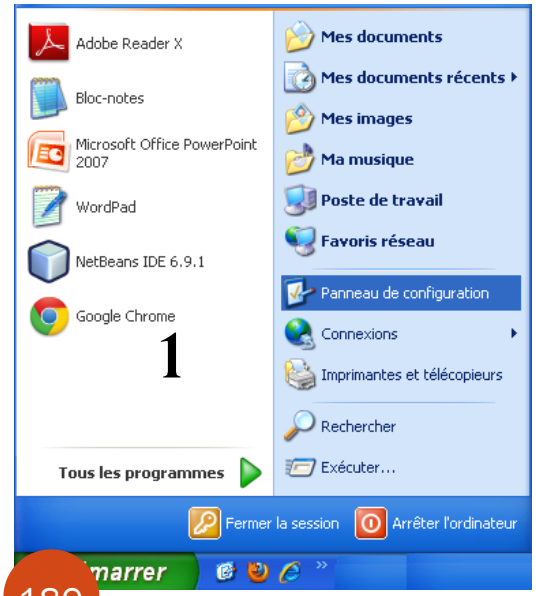
- BD: c: \ cours \ java \ bibliotheque.mdb (accdb)
  - Table: books
    - Fields:
      - LIV_BookCode
      - LIV_FirstNameAuthor
      - LIV_LastNameAuthor
      - LIV_Title
      - LIV_Category
      - LIV_ISBN

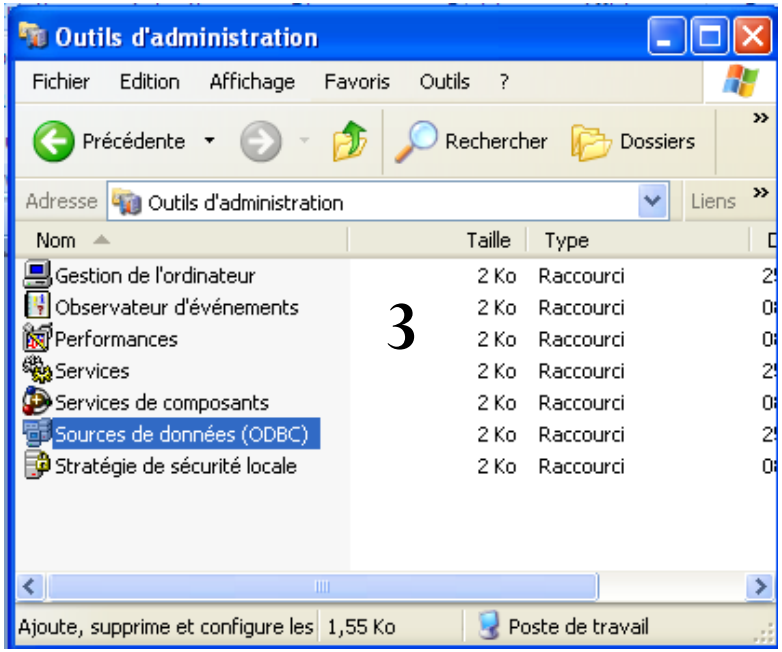
Livres

LIV_CodeLiv	LIV_NomAu	LIV_Prenom	LIV_Titre	LIV_Categor	LIV_ISBN	Ajouter un nouveau champ
1	Delannoy	Claude	Programmer en Java	Java	2212117485	
2	Karamti	Walid	Cours Java 2	Java	2015_2016	
(Nouv.)						

Adding an ODBC data source

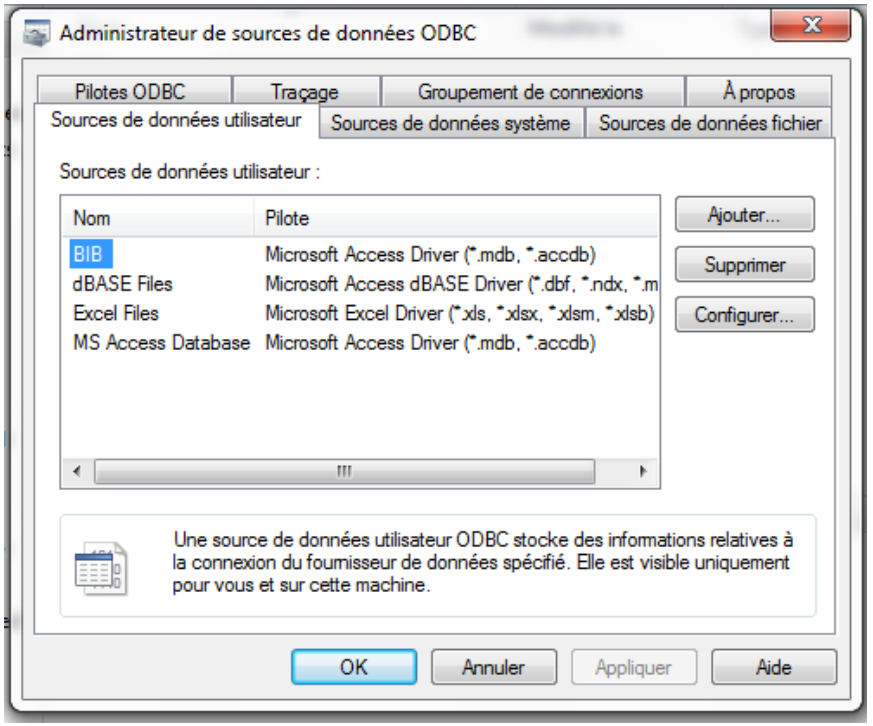






Adding an ODBC data source

4



- Add 5
  - Driver do Microsoft Access (* .mdb, *accdb)
    - Data Source Name (BIB)
    - Database path (c: \ ..... \ bibliotheque.mdb (accdb))

Steps to follow

- Import the java.sql package. *

- Load the driver into memory
- Ask the DriverManager create a connection to the database by specifying theUrl, the username and the password.

- Creation of an execution context: Statement
- Running the SQL query and receiving the results

- Closing the connection

Useful classes

Class name	Meaning
DriverManager	Driver manager, used to open a connection to the BD
Connection	Represents the physical connection to the DB
Statement	Execution context of the query
ResultSet	Represents the result of the SQL query valid in the case of SELECT

Driver loading

- Each database management server has its own driver
- Using dynamic class loading:
  - Class.forName("com.mysql.jdbc.Driver");
  - Class.forName("postgres95.pgDriver");
  - Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
  - Class.forName("com.microsoft.jdbc.sqlserver.SQLServerDriver");

Resource designation

- To establish a connection to a base, we will address the DriverManager an identification string of the resource sought.
- This string must follow a standard syntax that follows the syntax of URLs:

```
jdbc:sub_protocol: parameters
```

- Example:

```
String url = "jdbc: odbc: "+db_name; // ODBC

String url = "jdbc: mysql: // "+servername+ ":" + port + "/" +
  base; // MySQL
```

Obtaining Connection

- To obtain an instance of access to the database conforming to **java.sql.Connection** we must call:

```
Connection conx = DriverManager.getConnection(url);
```

- This object makes it possible to obtain information on the database and, above all, makes it possible to establish query contexts.

- Other forms of getConnection exist;

```
DriverManager.getConnection(url, "user", "password");
```

- if no driver is found the result will be **null**,

Obtaining a query context

- "Query context" objects (Statement) can be obtained as part of a connection.

```
// create a query context
Statement stmt;
try {
 stmt = conx.createStatement();
} catch (SQLException exc) {
 // handle exception
}
```

Obtaining a query context

- **3 types ofexecutions through the object Statement:**
  - ***executeQuery:*** for queries that return:
    - a ***ResultSet(SELECT)***
  - ***executeUpdate:*** for queries **:**
    - ***INSERT, UPDATE, DELETE, CREATE TABLE and DROP TABLE***
  - ***executed:*** for some rare cases (stored procedures)

Obtaining a query context

- **Example:**

```
int number = stmt.executeUpdate("DELETE from Clients WHERE
  id = 'CLI123' ");
```

- The method executeUpdate(sqlString) returns an integer that represents the number of records affected.
- The method executeQuery(sqlString) returns an object of type ResultSet which contains the result of a SELECT query

Exploitation of results

```
ResultSet rs = stmt.executeQuery("SELECT * from
  Clients");
```

- The type object ResultSet allows you to obtain information on the result and to browse the recordings:

```
int i = 0;
while (rs.next()) {
 String customer name = rs.getString(2);
 String id = rs.getString(1);
 table [i ++] = new Client (id, customer name);
}
```

Exploitation of results

- The columns of the record are numbered starting from 1.
  - For each column we can retrieve the result in an appropriate Java type using one of the methods getXXX()
    - getInt(): for integers
    - getString(): for character strings
    - Etc.

- It is also possible to search for a column by its name rather than by its serial number.

Mapping SQL data types in Java

SQL type	Type Java
CHAR, VARCHAR	String
NUMERIC, DECIMAL	java.math.BigDecimal
TINYINT	Byte
SMALLINT	shorts
INTEGER	int
BIGINT	long
REAL	float
FLOAT, DOUBLE	double
DATED	java.sql.Date
TIME	java.sql.Time

Connection to a database

```
import java.sql.Connection;
import java.sql.DriverManager;

.......

 try {
 Class.forName("fullname.of.the.class.driver");
 cnx = DriverManager.getConnection("BD URL");
 } catch (Exception e) {
 System.out.println ("Exception =" + e);
 }

.......
```

Connexion.java (MS Access, ODBC)

```
import java.sql.Connection;
import java.sql.DriverManager;



 try {
 Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
 cnx = DriverManager.getConnection("jdbc:odbc:BIB");
 } catch (Exception e) {
 System.out.println ("Exception =" + e);
 }
```

Connection.java (MySQL)

```
import java.sql.Connection;
import java.sql.DriverManager;



 try {
 Class.forName("com.mysql.jdbc.Driver");
 cnx = DriverManager.getConnection ("jdbc: mysql: // localhost: 3306
   /BD1","root","aabbcc");
 } catch (Exception e) {
 System.out.println ("Exception =" + e);
 }
```

Book.java

```
public class Book {
 // Define the properties
 private String title;
 private String category;
 private String isbn;
 private String author name;
 private String first name;
 private String code;
 public Book (String title, String category, String isbn, String
   author name, String first name) {
 this.title = title;
 this.category = category;
 this.isbn = isbn;
 this.authorname = author name;
 this.nameAuthor = first name;
 this.Code = calculationCode();
 }
```

Book.java

```
public void setTitle(String value) {
title = value;
}
public void setCategory(String value) {
category = value;
}
public void setIsbn(String value) {
isbn = value;
}
public void setNameAuthor(String value) {
author name = value;
}
public void setAuthorName(String value) {
first name = value;
 }
```

Book.java

```
public String getTitle() {
return title;
}
public String getCategory() {
return category;
}
public String getIsbn() {
return isbn;
}
public String getAuthorName() {
return author name;
}
public String getAuthorName() {
return first name;
}
```

Book.java

```
public String getCode() {code = calculationCode(); return code;}
private String calculationCode() {
String start name;
String start;
String startCategory;
int lengthIsbn;
String finIsbn;
start name = authorName.substring(0, 2);
start = firstnameAuthor.substring(0, 2);
startCategory = categorie.substring(0, 2);
lengthIsbn = isbn.length();
finIsbn = isbn.substring((lengthIsbn - 2), lengthIsbn);
return start name + start + startCategory + finIsbn;
}
}
```

Example: INSERT

```
import java.sql. *;
public class InsertionBook {
 public static void main (String [] args) {
 try {
 Delivered liv = new Book ("OOP in Java", "Programming",
  "0012344423", "Ali", "BEN SALAH");
 Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
 Connection con = DriverManager.getConnection("jdbc: odbc: BIB");

 Statement stmt = con.createStatement();
```

Example: INSERT

```
String myRequest = "INSERT INTO books VALUES ('" + liv.getCode()
  + "','" + liv.getNameAuthor() + "','" +
 liv.getNameAuthor() + "','" + liv.getTitle() + "','" +
  liv.getCategorie() + "','" + liv.getIsbn() + "')";


 int nb = stmt.executeUpdate(myRequest);
```

Example: INSERT

```
if (nb == 1)
System.out.println ("Insertion completed successfully.");
else
System.out.println ("No insertion.");
stmt.close();
con.close();
} catch (ClassNotFoundException cnfe) {
System.out.println ("Exception:" + cnfe.getMessage());
} catch (SQLException sqle) {
System.out.println ("Exception:" + sqle.getMessage());
}
}
}
```

Example: SELECT

```
.......
 String myRequest = "SELECT * FROM books";
 Statement stmt = con.createStatement();
 ResultSet rs = stmt.executeQuery(myRequest);
 while(rs.next()) {
 System.out.println (rs.getString(1) + "\ t" + rs.getString(2) +
   "\ t" + rs.getString(3)
 + "\ t" + rs.getString(4) + "\ t" + rs.getString(5) + "\ t" +
   rs.getString(6));
 }
 rs.close();
 stmt.close();
 con.close();
.......
```

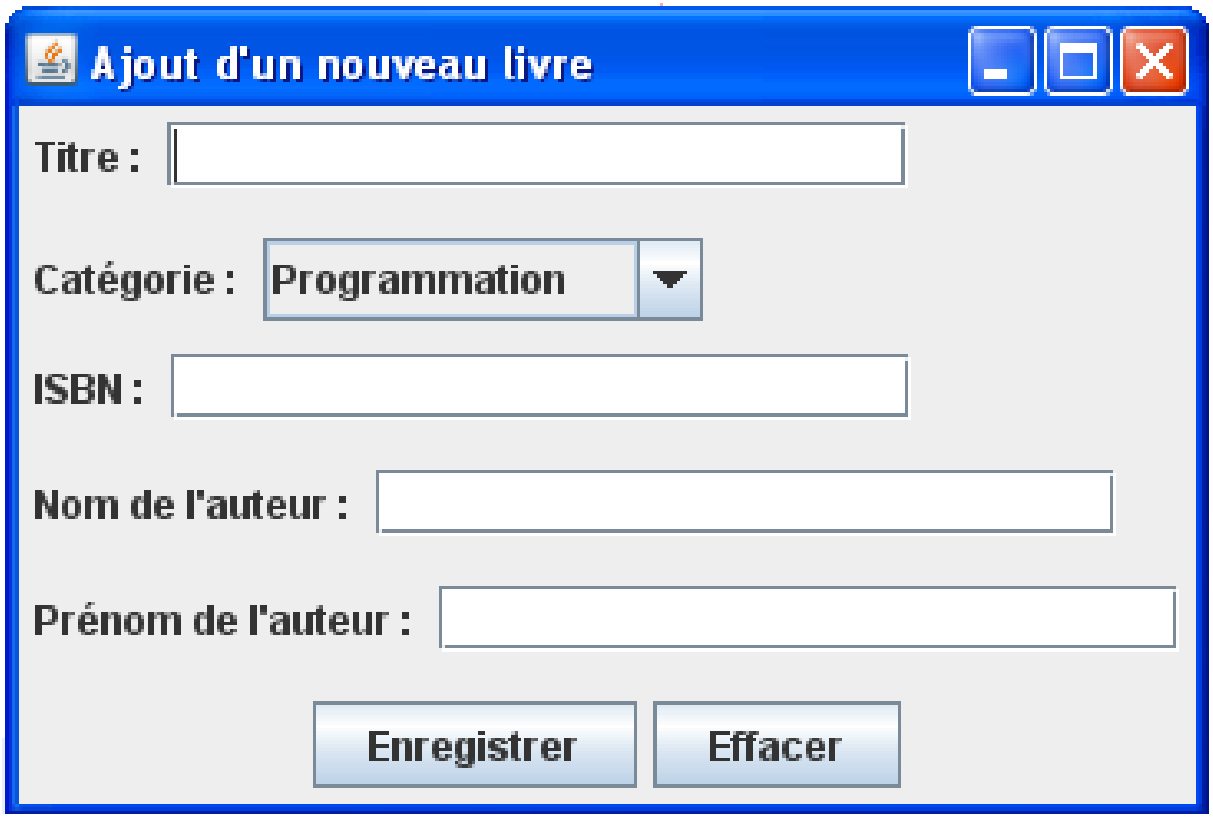
Example: UPDATE

```
.......
 String myRequest = "UPDATE books SET LIV_Title= 'Prog OO in Java
   'WHERE LIV_ISBN =' 112233 '";
 Statement stmt = con.createStatement();
 int nb = stmt.executeUpdate(myRequest);
 if (nb == 1) {
 System.out.println ("Modification successfully completed.");
 } else {
 System.out.println ("No modification.");
 }
 stmt.close();
 con.close();
.......
```

Example: DELETE

```
.......
 String myRequest = "DELETE FROM books WHERE LIV_ISBN =
   '112233'";
 Statement stmt = con.createStatement();
 int nb = stmt.executeUpdate(myRequest);
 if (nb == 1) {
 System.out.println ("Deletion completed successfully.");
 } else {
 System.out.println ("No deletion.");
 }
 stmt.close();
 con.close();
.......
```

Application: library management



Application: library management

- Write a Connection class that has an attribute cnx Of type Connection.
  - Declare a constructor that allows you to initialize the object cnx and create a connection to our BIB data source.
  - Add method getCnx()
- In the book class, add a save instance method which allows you to save the current object in the DB.

Chapter 5
Threads in Java

Dr Hedia ZARDI

H.zardi@qu.edu.sa

Introduction

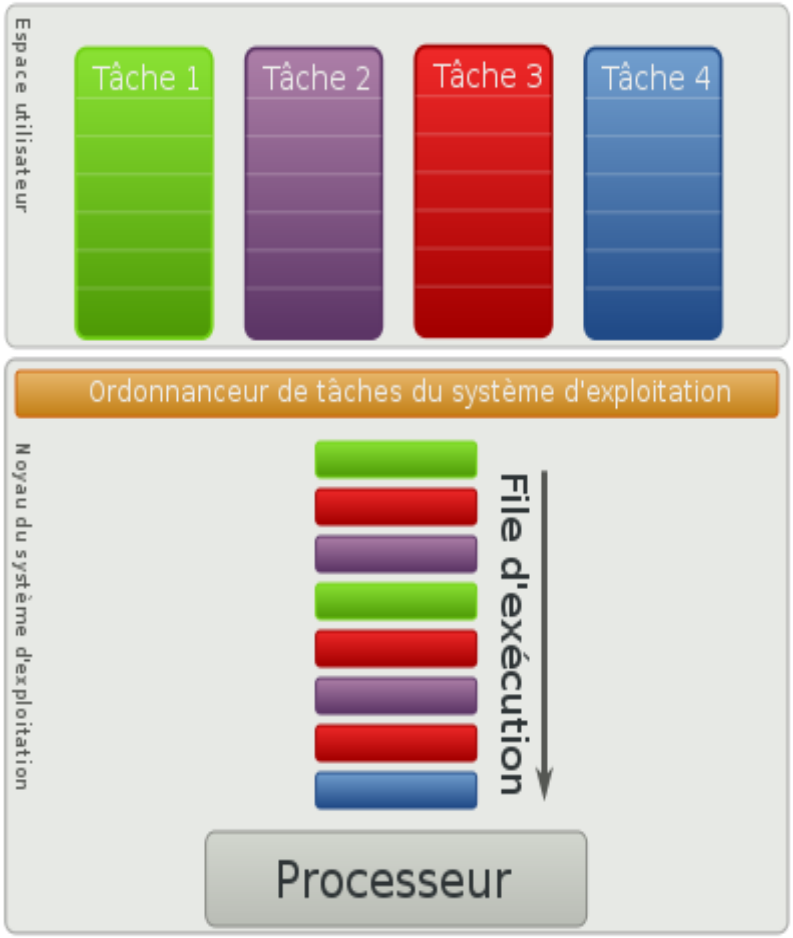
How does an operating system allow to have several tasks that run in same time?

Even though it only has one processor for actually perform a task, he is able to give an "illusion" of parallelism.

Introduction

- In a modern, multitasking operating system there are several programs (processes) that "run" at the same time. Each one executes a certain code and has its own data.

- At a finer level, we find the notion of Thread ("light" processes) which are managed within a process itself and which share the same data.

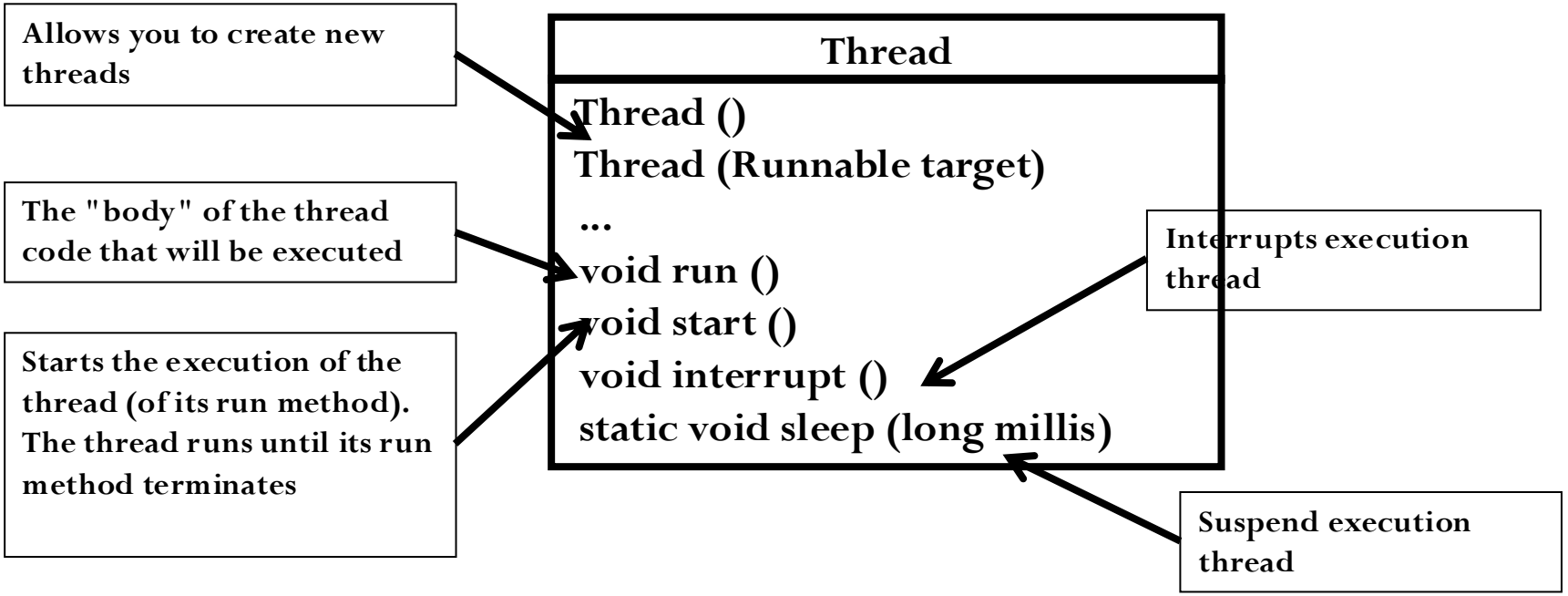


What is a Thread?

- Thread: lightweight process
  - similar to a process in that a thread and an executing program both represent a flow of sequential control
    - A thread has its own execution stack and program counter (execution context)
  - Lightweight: because it runs in the context of a program and uses the resources allocated for that program and its environment
- The java virtual machine allows a java application to define several streams (threads) of execution running concurrently

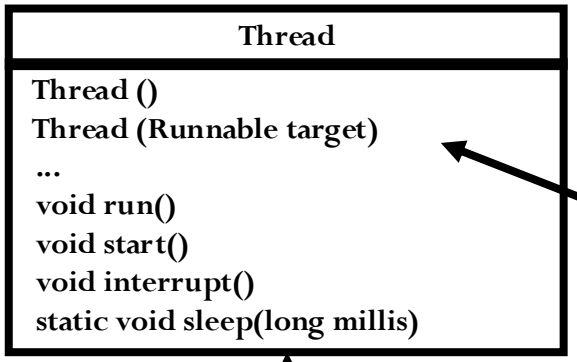
Definition of Threads

- Class **java.lang.Thread**: the class at the center of thread management in the Java API

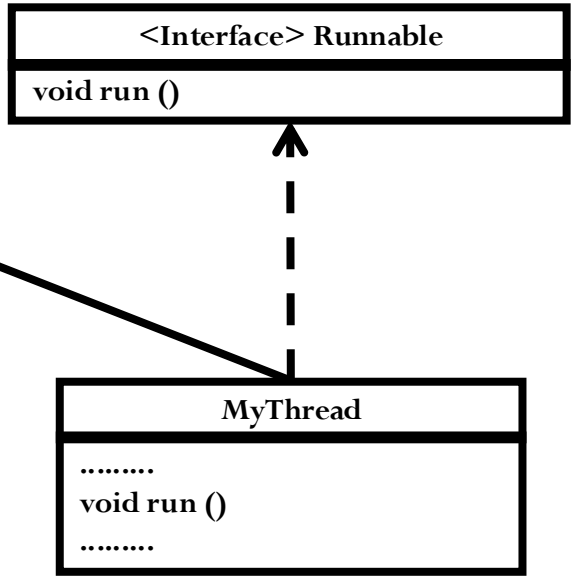


Two ways of defining Thread

- Extend Thread and Redefine run



- Pass an object implementing the interface as a parameter of the constructor Runnable



But in this case MyThread cannot inherit from another class (simple inheritance in Java)

Create a new thread (1)

```
public class MonThread extends Thread {
    public MonThread() {
    }
    public void run() {
        System.out.println("Hello");
    }
    public static void main(String[] args) {
        MonThread t = new MonThread();

    }
}
```

Allows you to start the execution.

The Thread is created corn it is not launched to run !!!!

Creating a Thread Instance

Create a new thread (2)

```
public class MaTache implements Runnable {
    public MaTache() {
    }
    public void run() {
        System.out.println("Hello");
    }
    public static void main(String[] args) {
        MaTache tache = new MaTache();
        Thread t = new Thread(tache);
        t.start();
    }
}
```

Start the execution of Thread

Creating a Thread Instance

Creation of an instance of Runnable

Example

```
public class Simple Counter {
 private String name;
 private int max;
 public Simple Counter(String name, int max) {
 this.name = name;
 this.max = max;
 }
 public void to start up() {
 for (inti = 0; i <max; i ++) {
 System.out.println (name + ":" + i);
 }
 }
}
```

```
Counter 1: 0
Counter 1: 1
Counter 1: 2
Counter 2: 0
Counter 2: 1
Counter 2: 2
```

```
public class SimpleTest {
 public static void main (String [] args) {
 new Simple Counter("Counter 1", 3).to start up();
 new Simple Counter("Counter 2", 3).to start up();
 }
}
```

Example

```
public class ThreadCounter implements Runnable{
 private String name;
 private int max;
 public ThreadCounter(String name, int max) {
 this.name = name;
 this.max = max;
 }
 public void run() {
 for (int i = 0; i <max; i ++) {
 System.out.println (name + ":" + i);
 }
 }
}
```

```
Counter 1: 0
Counter 1: 1
Counter 1: 2
Counter 2: 0
Counter 2: 1
Counter 2: 2
```

```
public class TestCounterThread {
 public static void main (String [] args) {

 new Thread (new ThreadCounter ("Counter 1", 3)). start ();

 new Thread (new ThreadCounter ("Counter 2", 3)). start ();
 }
}
```

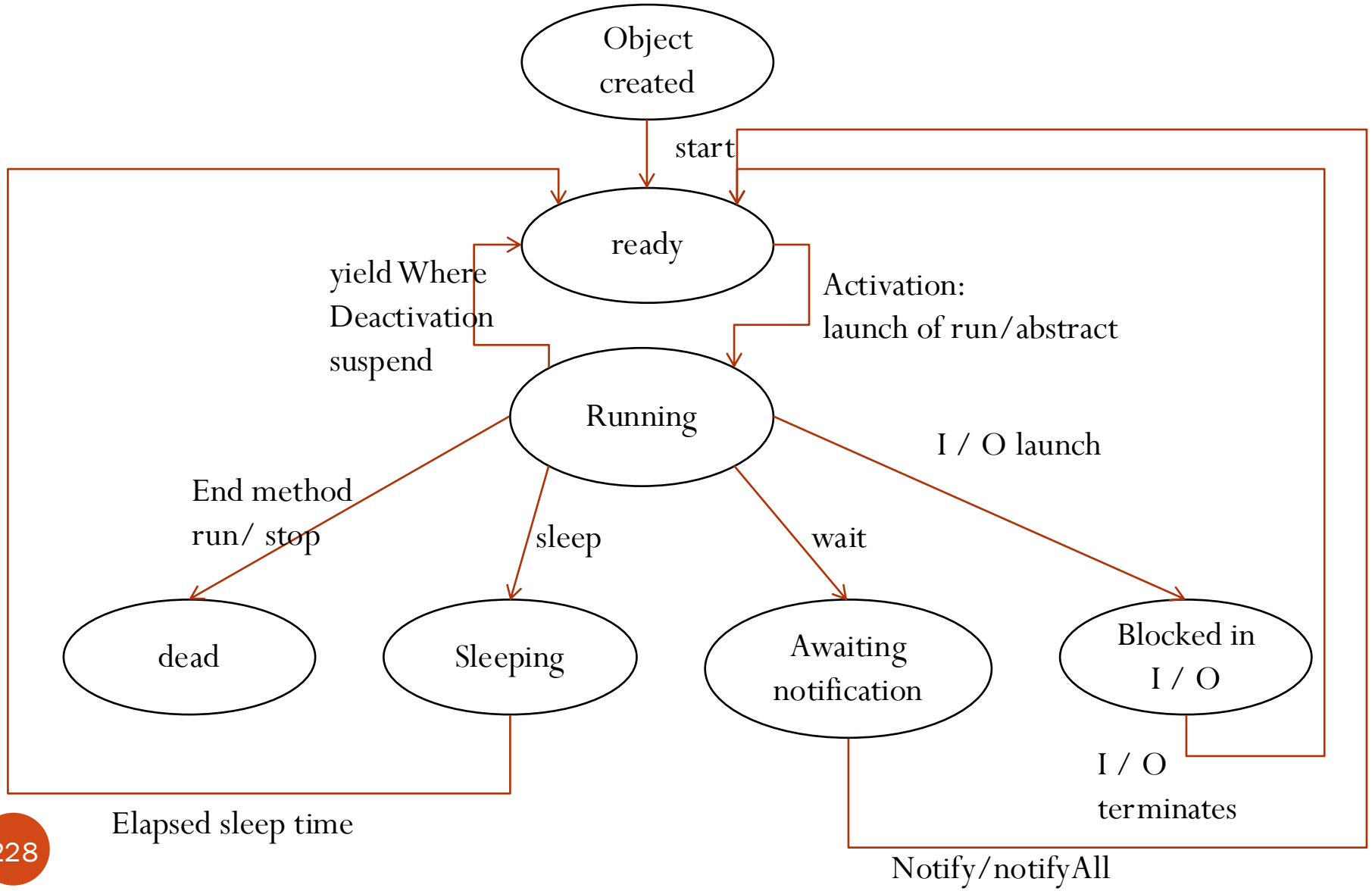
Example

```
public class ThreadCounter implements Runnable{
 private String name;
 private int max;
 private long waiting;
 public ThreadCounter(String name, int max, long
waiting) {
 this.name = name;
 this.max = max;
 this.waiting= waiting;
 }
 public void run() {
 try{
 for (inti = 0; i <max; i ++) {
 System.out.println (name + ":" + i);
 sleep(waiting);
 }
 }
 catch (InterruptedException e)
 {
 e.printStackTrace();
 }
}
```

```
Counter 1: 0
Counter 2: 0
Counter 1: 1
Counter 2: 1
Counter 1: 2
Counter 2: 2
```

```
public class TestCounterThread {
 public static void main (String [] args) {
 new Thread (new ThreadCounter("Counter 1 ", 3, 10)).start ();
 new Thread (new ThreadCounter("Counter 2 ", 3, 5)).start ();
 }
}
```

Life cycle


Object created
start
ready
yield Where
Deactivation
suspend
Activation:
launch of run/abstract
Running
I / O launch
End method
run/ stop
sleep
wait
dead
Sleeping
Awaiting notification
Blocked in I / O
I / O terminates
Elapsed sleep time
Notify/notifyAll

The states of a thread

- **Created:**
  - like any Java object
  - … but is not yet active
- **Active:**
  - after creation it is activated by start() which launches run().
  - It is then added to the list of active threads to be executed by the time-sharing OS
  - Can return to this state after a abstract() or one notify().

The states of a thread

- **Asleep or blocked:**
  - after sleep(): asleep for a time interval (ms)
  - suspend () puts the Thread to sleep but abstract() reactivates it
  - a blocking input / output (file opening, keyboard input) puts to sleep and wakes up a Thread

- **Dead :**
  - if stop () is called explicitly
  - when run() has finished its execution

The join () method

- The instance method `join ()` allows the executing Thread to wait for the target Thread to finish executing.

The join method

```
public class CompteurChaine extends CompteurThread {
    private Thread autre;
```

dy: Counter 2
unter 1: 0
counter
unter 1: 2
unter 2: 0
unter 2: 1
unter 2: 2

}

The join method

```
public class CompteurChaine extends CompteurThread {
    private Thread autre;
    public CompteurChaine(String nom, int max, Thread autre) {
        super(nom, max);
        this.autre = autre;
    }
    public void run() {
        System.out.println("prêt:" + getNom());
        autre.start();
        try {
            autre.join();
        } catch (InterruptedException ie) {
            System.out.println(ie.getMessage());
        }
        super.run();
    }
    public static void main(String[] args) {
        Thread tc = new Thread(new CompteurThread("Compteur 1", 3));
        Thread tcc = new Thread(new CompteurChaine("Compteur 2", 3, tc));
        tcc.start();
    }
}
```

```
ready: Counter 2
Counter 1: 0
Counter 1: 1
Counter 1: 2
Counter 2: 0
Counter 2: 1
Counter 2: 2
```

The yield () method

- The class method `Thread.yield()` allows the executing thread to ask the Scheduler ***kindly*** bring it to the ready state.

The priorities

- Principles :
  - Java allows you to modify the priorities (absolute levels) of Threads by the method setPriority()
  - By default, each new Thread has the same priority as the Thread that created it
  - **Recall :** only active threads can be executed and therefore access the CPU
  - The JVM chooses to run the active Thread that has the highest priority: priority-based scheduling
  - if several Threads have the same priority, the JVM distributes the CPU time equally (time slicing) between all: round-robin scheduling (the turnstile).

The priorities

- The methods :
  - setPriority(int): sets the priority of the receiver.
    - the parameter must belong to: [MIN_PRIORITY, MAX_PRIORITY]
    - otherwise IllegalArgumentException is up
  - int getPriority(): to know the priority of a Thread
  - NORM_PRIORITY: gives the "normal" priority level

Thread coordination

- The advantage of threads over processes is that they belong to the same program: they share the same objects.

- This advantage is sometimes accompanied by constraints:
  - It should be avoided that two threads can access the same object at the same time (**access competition**).
  - A thread will have to wait until another has completed some work on an object before it can continue its execution itself.

Thread coordination

- The first problem is solved by the use of mutually exclusive methods which are often called "synchronized methods".

- The second problem will be solved by waiting and notification mechanisms implemented using the methods wait and notify.

Access competition

- The problem: common workspace, no "private memory" for each thread:
  - disadvantage: simultaneous access to the same resource

  Exclusive access to an object must be guaranteed during the execution of one or more instructions

- To do so: the keyword **synchronized** allows you to manage concurrency:
  - of a method
  - of an object
  - or an instruction (or a block)

The synchronization

- **Based on the technique of mutual exclusion:**
  - with each Java object is associated a "lock" managed by the thread when a method (or an object) synchronized is accessed.
  - guarantees exclusive access to a resource (the critical section) during the execution of a piece of code.
- **A critical section:**
  - a method : declaration preceded by synchronized
  - an instruction (or a block): preceded by synchronized
  - an object : declare it synchronized
- **Beware of interlocking !!**

Use synchronized

- To manage concurrency of access to a method:
  - if one thread executes this method on an object, another thread cannot execute it for the same object
  - however, it can execute this method for another object

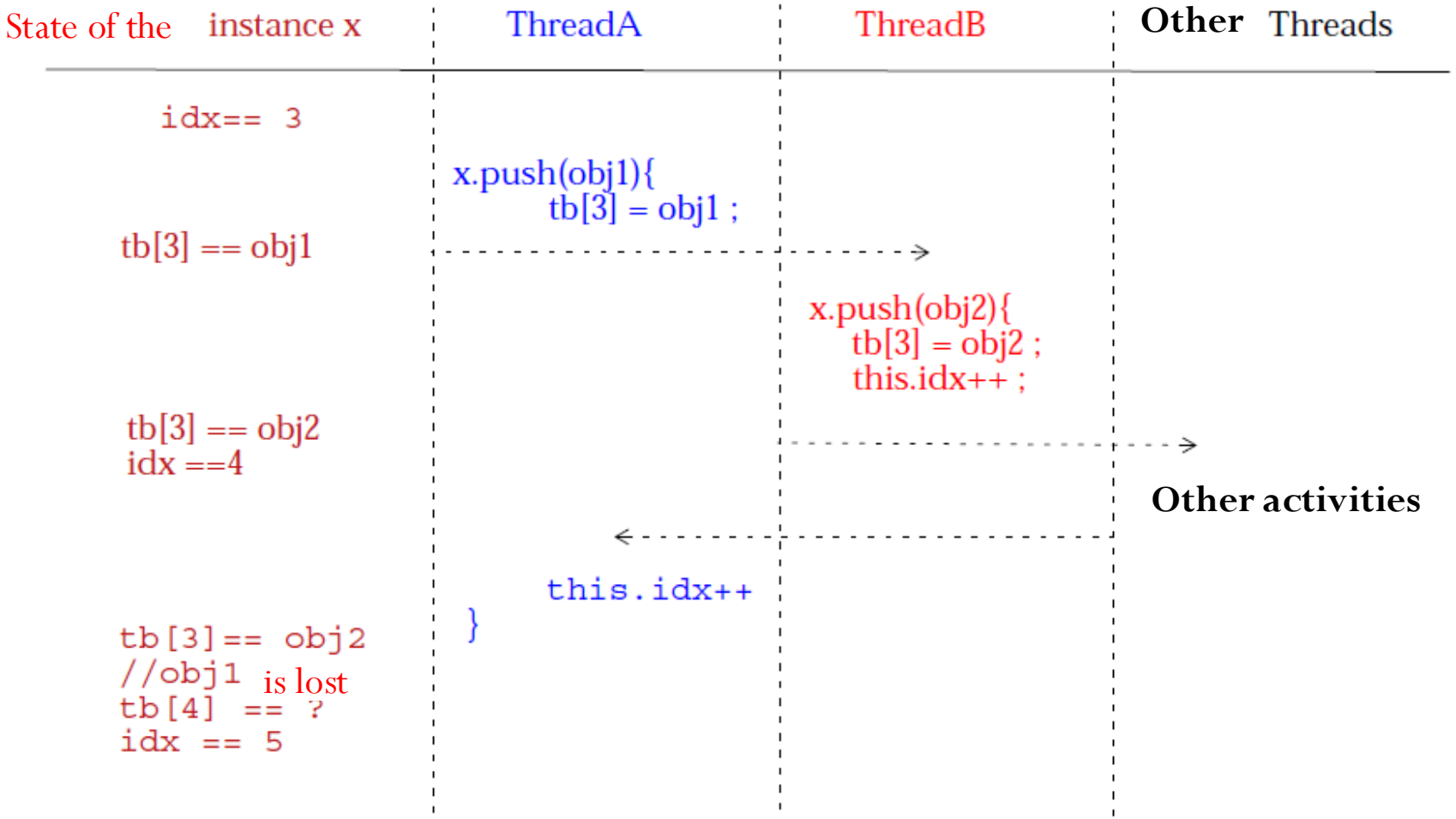
```
public synchronized void myMethod() {...}
```

- To control access to an object:

```
public void myMethod() {...
 synchronized(object) {
 object.methode();
 }
```

  - Access to the object passed as a parameter of synchronized(Object) is reserved for a single thread.

Use synchronized: Example



Use synchronized: Example

```
class MyPile {
 // ... miscellaneous code including definition of
MAX
 private Object [] tb = new Object [MAX];
 private int idx = 0;

 public void push (Object obj) {
 // we assume that we are processing the pb size
 tb[idx] = obj;
 this.idx ++;
 }

 public Object pop () {
 // we assume that we are preventing unstacking an
empty stack (exception)
 this.idx--;
 return tb[idx];
 }
}
```

Use synchronized: Example

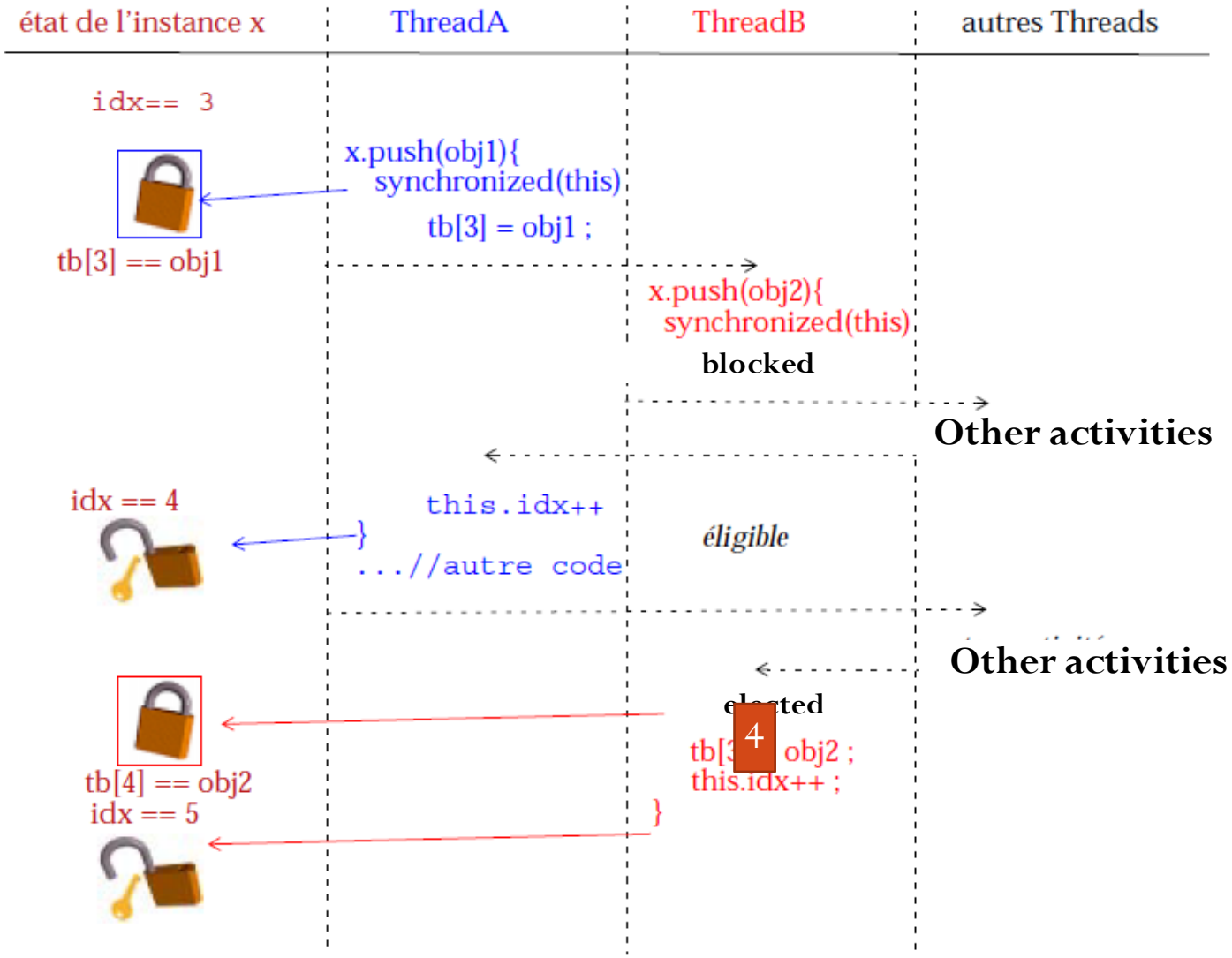
```
class MyPile {
 // ... miscellaneous code including
definition of MAX
 private Object [] tb = new Object [MAX];
 private int idx = 0;

 public void push (Object obj) {
 // we assume that we are processing the pb
size
 synchronized(this) {

 tb[idx] = obj;
 this.idx ++;

 }
 }
}
```

Use synchronized: Example



The deadlock

- The use of locks on objects can lead to a deadlock situation which can be defined as follows:
  - The thread t1 o1 owns the lock of the object and it waits for the lock of the object o2,
  - Thread t2 owns the lock of object o2 and waits for the lock of object o1.
- Java is not able to detect this kind of situation and it is up to the programmer to handle this task.

Waiting and notification

- Sometimes you need to coordinate the execution of threads
  - A thread that must wait for another to perform a certain task to continue its execution.
- Here again, Java offers a mechanism based on the object and on the synchronized methods that we have just studied:
  - A synchronized method can call the method wait of the object for which it has the lock, which has the effect of:
    - return the lock to the environment which can assign it to another synchronized method,
    - put the corresponding thread on hold; more than one thread can be on hold.
  - A synchronized method can call the method notifyAll of an object to notify all threads waiting on that object and give them the opportunity to run.

The "ThreadGroup"

To control multiple threads

- Several processes (Thread) can run at the same time, it would be useful to be able to handle them as a single entity
  - to suspend them
  - to stop them, …

Java offers this possibility through the use of thread groups:

`java.lang.ThreadGroup`

- we group a named set of threads
- they are controlled as a single unit

Thread groups

- A tree structure:
  - the class `ThreadGroup` allows you to build a tree structure of Threads and ThreadGroups
  - it gives classical methods of recursive manipulation of a set of threads: `suspend (), stop (), abstract(),...`
  - and specific methods: `setMaxPriority(),...`
- Functioning :
  - the JVM creates at least one group of threads named main
  - by default, a thread belongs to the same group as the one that created it (its father)
  - `getThreadGroup()` : to know your group

Creating a thread group

- To create a process group:

```
ThreadGroup group = new ThreadGroup("My group");
Thread p1 = new Thread (group, "P1");
Thread p2 = new Thread (group, "P2");
Thread p3 = new Thread (group, "P3");
```

- You can create subgroups of threads for the creation of sophisticated process trees
  - from ThreadGroup contain ThreadGroup
  - threads can be at the same level as ThreadGroup